

Generator-Aware Positive Row Scaling for Block-Structured Engineering Mixed-Integer Programs

Mathias Rodríguez Castro^{a,*}

^a*Facultad de Ingeniería, Universidad de la República, Montevideo, Uruguay*

Abstract

Generated engineering MIPs are usually exported to solvers as flat coefficient matrices, but the generator that creates them knows more: which constraints are local to physical or operational components, which variables belong to those components, and which rows couple components through system-wide balances. This paper studies whether that pre-export metadata can be used as numerical information for positive row scaling, without changing the mixed-integer program.

We propose a generator-aware row-scaling framework for block-structured engineering MIPs. The method multiplies complete constraints, including right-hand sides, by strictly positive factors, so the feasible set, objective function, and integer restrictions are preserved exactly. The contribution is not row scaling in isolation, but the use of generator metadata to choose row factors in a local-block-coupling order: local component rows are scaled first, representative block scales are then computed, and coupling rows are finally scaled relative to the blocks they connect. Idealized conditioning results for block-diagonal and block-coupled matrices explain why this order separates local scale imbalance from coupling imbalance.

The method is evaluated on a short-term hydrothermal dispatch platform that generates modular MIPs with heterogeneous physical and economic units. The application is used as a demanding generated-engineering testbed, not because the scaling rules depend on hydro-specific equations: the required metadata are only row roles, block ownership, and coupling incidence. Two generator-aware variants are compared with the unscaled formulation and Ruiz-type baselines across three instance classes, two commercial solvers, and two optimality tolerances. A controlled synthetic benchmark injects local and coupling imbalance by construction and serves as a mechanism check for conditioning recovery and original-scale equivalence. The evidence supports three scoped claims. Positive row scaling is algebraically exact; generator-aware scaling robustly improves solver-facing numerical diagnostics in this testbed; and performance gains are solver dependent. Under Gurobi, the improved representation often reduces robust runtime or deterministic effort, whereas under CPLEX 22.1.2.0 it does not yield a robust runtime gain. The method should therefore be read as a structure-aware numerical preprocessing layer for generated block-structured engineering MIPs, not as a hydrothermal heuristic or a solver-independent acceleration theorem.

Keywords: Numerical scaling, mixed-integer programming, generated formulations, block-structured optimization, engineering optimization, numerical preprocessing, hydrothermal dispatch

1. Introduction

A model generator and a solver see different representations of the same optimization problem. The generator builds the model from components: hydro plants, thermal units, market rules, resource limits, demand balances, and discrete operating decisions. While doing so, it knows which rows are local to one component, which variables that component owns, and which constraints link several components. The solver receives the exported version: rows, columns, coefficients, bounds, and tolerances. In that exported matrix, much of the generator's structural knowledge has disappeared.

This loss matters because numerical representation affects computation. Two formulations can describe the same feasible set and objective, yet lead a solver through different presolve reductions, relaxation bases, cuts, incumbent searches,

branching decisions, and feasibility checks. Generic scaling routines can inspect coefficient magnitudes after export. They usually cannot tell whether a row is a local turbine restriction, a thermal limit, or a system-wide balance. The generator can make that distinction before export.

This distinction is not made obsolete by solver-internal scaling. Commercial solvers already contain sophisticated numerical preprocessing, and the experiments below leave those mechanisms enabled. The question studied here is complementary: can an external preprocessing layer supply structural numerical information before the solver sees the matrix as an anonymous list of rows and columns? A positive answer would not replace solver scaling; it would change the representation on which the solver's own pipeline starts.

The resulting question is simple: can generator-level block information guide positive row scalings that improve the solver-facing representation without changing the MIP? The question is not whether positive row scaling is valid. Multiplying a complete

*Corresponding author.

Email address: mathiasr@fing.edu.uy (Mathias Rodríguez Castro)

constraint by a strictly positive scalar preserves the feasible set. The question is whether the scaling factors should depend on information that the generator still has: local rows, component blocks, and coupling rows. In this sense, the paper treats pre-export metadata as numerical information rather than only as modeling bookkeeping.

We answer this question with a generator-aware row-scaling framework. The method scales only whole constraints, including their right-hand sides. Hence it preserves the original mixed-integer optimization problem. Its novelty lies in the order and source of the scales, not in claiming that row scaling is new. The generator first scales local component rows. It then summarizes the scale of each locally scaled block. Finally, it scales global coupling rows with the scales of the blocks they connect. Throughout the paper, this local–block–coupling order is the main object of study.

The paper contributes four elements: an algebraically exact positive row-scaling layer for generated block-structured MIPs; a generator-aware rule that uses row roles, component ownership, and coupling incidence; idealized conditioning arguments that motivate the local–block–coupling order; and an empirical evaluation that separates numerical representation from solver-dependent performance.

The numerical rationale is also local–block–coupling. In an ideal block-diagonal matrix, scalar block scaling can reduce scale separation between blocks, but it cannot repair the intrinsic conditioning of a block. Once coupling rows are added, their relevant magnitude is not just their raw coefficient size; it is their size relative to the scaled local blocks. Idealized conditioning results make this distinction precise. They justify the architecture of the preprocessing rule, but they do not claim to predict mixed-integer runtime.

We test the method on short-term hydrothermal dispatch. The methodological contribution is not a new hydrothermal formulation; rather, the dispatch platform is a demanding testbed for the preprocessing idea. It naturally creates local hydro, thermal, market, shortage, and demand modules, then links them through global balance equations. The coefficients mix water volumes, turbine flows, power outputs, generation costs, shortage penalties, and demand balances. The testbed therefore combines engineering structure with numerical scale heterogeneity.

The intended level of generality is the generated block-structured MIP, not the hydrothermal equations themselves. The preprocessing layer requires only metadata that many modular engineering generators already possess: which rows are local, which block owns a variable or constraint, and which global rows couple blocks. Reservoir dynamics, turbine curves, and market-cost details determine the experimental instances, but they do not enter the definition of the scaling rule. This separation is important for scope: the hydrothermal model supplies a realistic stress test, while the proposed numerical layer targets any generated engineering MIP with local components and sparse global coupling.

The hydrothermal formulation used in the experiments descends from models for the Río Negro hydroelectric complex and extensions to hydrothermal dispatch with Salto Grande,

developed by Risso and coauthors, together with a modular C++ implementation used as the experimental platform [1, 2, 3]. We take the formulation as given and insert the proposed preprocessing layer after model generation and before solver invocation.

The paper makes three claims of different strength. The algebraic claim is exact: positive row scaling preserves the MIP. The numerical claim is empirical: in the tested hydrothermal instances, generator-aware scaling improves solver-facing conditioning measures. The performance claim is conditional: under Gurobi, the improved representation often reduces robust runtime or deterministic effort; under CPLEX 22.1.2.0, the same external transformations do not produce a robust runtime gain. Thus the method improves representation, while acceleration depends on how the solver uses that representation.

The rest of the paper follows this chain of evidence. [Section 2](#) identifies the research niche. [Section 3](#) defines the generated model structure and testbed. [Section 4](#) gives the scaling method. [Section 5](#) explains how the experiments test the claims. [Section 6](#) reports numerical and runtime evidence. [Section 7](#) interprets that evidence and answers likely objections. [Section 8](#) states the main limitations and reproducibility implications, and [section 9](#) summarizes the contribution.

2. Related work

The literature establishes the setting for the paper but leaves a specific niche. Hydrothermal-dispatch models provide realistic generated engineering MIPs with heterogeneous physical units and global balance constraints. Numerical-scaling and solver-numerics work explain why coefficient magnitudes matter, but their standard tools operate mostly on the exported matrix. Formulation-quality and modeling-system practice show that a model’s algebraic representation can be as important as the underlying mathematical set. The missing argument is therefore not that scaling is useful in general. It is narrower: when a generator still knows the role of each row and the block ownership of each variable, that metadata can be used as numerical information before export, and the resulting transformation must be evaluated separately as an algebraic equivalence, a numerical representation change, and a solver-dependent performance intervention.

2.1. Hydrothermal dispatch and MIP formulations

Hydrothermal dispatch has been studied with dynamic programming, stochastic dual dynamic programming, nonlinear programming, and mathematical programming. Classical references emphasize the trade-off between thermal generation costs, hydro opportunity costs, demand satisfaction, and intertemporal reservoir management [4, 5]. In short-term settings, MIP formulations are attractive because they incorporate discrete decisions, piecewise-linear approximations, local operating constraints, and system-wide balance equations in a single model [6].

This literature motivates the experimental domain but not the novelty claim. The paper does not propose a new hydrothermal

dispatch formulation, a new water-value model, or a new market-clearing rule. The dispatch model is used because it is a representative generated engineering MIP: local physical modules are created separately, their variables have heterogeneous units, and a smaller set of global rows links the modules through system-wide balances. These are precisely the structural features exploited by the preprocessing layer.

2.2. Dispatch models used in this study

The computational experiments use a reference formulation from a research line on MIP-based dispatch models for the Uruguayan power system. Risso et al. introduced a mixed combinatorial optimization model for the Río Negro hydroelectric complex [1] and extended the modeling line to hydrothermal dispatch for the Uruguay River hydroelectric plant, with emphasis on Salto Grande and coupled hydrothermal operation [2]. The implementation used here builds on a modular C++ version of the Río Negro dispatch model [3].

This provenance fixes the scope of the experiments. The dispatch formulation, data pipeline, and physical components are taken as given. The proposed method acts after the model has been generated and before the solver is called. Consequently, any observed difference between variants is attributed to an algebraically equivalent representation of the same generated MIP, not to a modified dispatch model.

2.3. Numerical scaling and solver-facing representation

Scaling is a standard preprocessing operation in linear and mixed-integer optimization. Multiplying a constraint by a positive factor preserves the feasible region but changes the numerical magnitudes passed to the solver. These magnitudes can affect feasibility tolerances, relaxation stability, presolve transformations, cut generation, basis quality, and the numerical behavior of LP relaxations. Practical guidelines for difficult LP and MIP models warn that large coefficient ranges and poor scaling can harm solver performance even when the mathematical formulation is correct [7, 8].

This paper uses that observation in a deliberately restricted way. It does not claim that coefficient compression alone is an objective, nor that a smaller coefficient range must produce a faster branch-and-bound search. A MIP solver is a pipeline: presolve, internal scaling, root relaxation, cuts, heuristics, incumbent updates, branching, node processing, and termination criteria can all respond differently to the same external scaling. For that reason, the experiments distinguish exported coefficient-range ratios from solver-level conditioning diagnostics and from runtime metrics. This distinction is central to the paper’s interpretation of the Gurobi–CPLEX contrast.

2.4. Generic equilibration and its limitation for generated models

Generic equilibration procedures balance rows and columns according to matrix norms. Ruiz-type scaling is a representative method: it iteratively equilibrates row and column norms through diagonal transformations [9]. Such methods are useful because they are model-agnostic and can be applied after export.

That strength is also their limitation in the setting considered here. A flat-matrix method can see coefficient magnitudes and sparsity, but it is not told whether a row is a local turbine restriction, a thermal bound, a market-cost relation, a shortage constraint, or a system-wide balance.

The proposed method keeps the model monolithic, as generic scaling does, but changes the source of the scaling information. Local rows are first treated within their generating component; the scaled local rows then define block-level representative scales; coupling rows are finally scaled relative to the blocks they connect. This is not a replacement for solver-internal scaling or generic equilibration. It is an external, auditable layer that uses generator metadata before that metadata is lost in the exported coefficient matrix.

2.5. Formulation quality, generated structure, and decomposition

The MIP literature also stresses that formulation quality matters: two algebraically related formulations can describe the same modeling intention but lead to different relaxations, numerics, and search behavior [6]. The present work is adjacent to that tradition but changes a different object. It does not strengthen the formulation, add valid inequalities, alter variables, or change the feasible set. It changes only the positive row multipliers applied to complete constraints.

The method is also adjacent to structural decomposition. Like Dantzig–Wolfe, Benders, or Lagrangian approaches, it recognizes local blocks and global linking rows. Unlike those methods, it does not decompose the optimization problem, introduce a master problem, or exchange prices or cuts between subproblems. The solver still receives one monolithic MIP. The use of block structure is numerical rather than algorithmic decomposition: block metadata guide the representation presented to the solver.

2.6. Remaining niche

The resulting contribution is deliberately narrow. It tests whether local rows, block scales, and coupling rows should be treated as different numerical objects before a generated engineering MIP is exported to a commercial solver. The benchmark standard for the performance comparison is not a single runtime average but distributional evidence over heterogeneous instances; for this reason the paper uses performance profiles [10], paired tests, deterministic-effort metrics, and solver-level conditioning diagnostics. The claim supported by the study is therefore a scoped one: generator-aware positive row scaling can improve the solver-facing representation of generated block-structured engineering MIPs, while acceleration remains conditional on the solver pipeline.

3. Materials and methods: generated modular formulation and testbed

We now define the objects that the scaling rule changes and the objects it must leave unchanged. The generator supplies the block structure; the preprocessing layer changes the row

magnitudes; the solver then receives an algebraically equivalent MIP. The dispatch model itself is not new. What matters here is the structure that the generator exposes and the original scale in which all feasibility and objective checks are evaluated after the solve. We consider generated modular MIP instances formulated as

$$\begin{aligned} \min_x \quad & c^\top x \\ \text{s.t.} \quad & Ax \leq b, \\ & x_j \in \mathbb{Z}, \quad j \in \mathcal{I}, \\ & x_j \in \mathbb{R}, \quad j \notin \mathcal{I}. \end{aligned} \quad (1)$$

In the general generated setting, the vector x collects all variables created by the modules, while A and b contain both local component constraints and global linking constraints. In the hydrothermal testbed used here, x includes generation, reservoir, flow, spillage, shortage, and operating variables. Equalities are represented either explicitly or by pairs of inequalities, depending on the solver interface.

The notation is intentionally application-agnostic. The preprocessing layer does not use hydrological identities, thermal-unit semantics, or economic labels directly. It uses the generator's structural map: row role, block ownership, and coupling incidence. The hydrothermal variables listed above explain the physical origin of the coefficients in the testbed; they are not assumptions required by the scaling method.

3.1. Hydrothermal dispatch testbed and scope

The computational testbed uses the reference model generated by the modular dispatch platform developed around Río Negro and Salto Grande hydrothermal dispatch models [1, 2, 3]. The application is useful because it exhibits the target structure: local modules with heterogeneous physical units and global rows that couple those modules through system-wide balances. The hydraulic, thermal, and market-cost constraints are those of the reference model and its implementation.

The paper therefore separates the preprocessing idea from the application that tests it. The proposed method acts on the generated algebraic model; it does not modify the physical dispatch formulation. All preprocessing transformations must preserve the feasible set and must allow objective values, constraint violations, and operational variables to be interpreted in the original scale after solution. In particular, the method never uses a hydro-specific rule such as reservoir priority, turbine efficiency, or unit commitment logic to choose scaling factors. Those details generate the rows; after generation, only their structural role in the block-coupled MIP is used.

3.2. Block structure

The method targets models generated by components or agents. Each component $i \in \mathcal{B}$ creates local variables x_i , contributes local constraints $A_i x_i \leq b_i$, and appears in a smaller number of global constraints. Let $\mathcal{R}_i$ denote the local row index set of component i . In the hydrothermal testbed, these components include hydro, thermal, market, and demand modules. If u_{loc} stacks all local variables and g denotes an

aggregate coupling variable, the row structure can be written schematically as

$$\begin{bmatrix} D & 0 \\ 0 & I_T \\ L & -I_T \end{bmatrix} \begin{bmatrix} u_{\text{loc}} \\ g \end{bmatrix} \sim \begin{bmatrix} b \\ d \\ 0 \end{bmatrix}, \quad D = \text{diag}(A_1, \dots, A_B). \quad (2)$$

The first block contains local component restrictions. The second block represents an aggregate system restriction. The third block contains coupling equations that link local component decisions with the aggregate system variable. In the dispatch application, these rows correspond to demand and balance relationships. Equivalently, after row and column permutations, the coefficient matrix has the block form

$$A = \begin{bmatrix} A_1 & 0 & \cdots & 0 \\ 0 & A_2 & \cdots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \cdots & A_B \\ L_1 & L_2 & \cdots & L_B \end{bmatrix}, \quad b = \begin{bmatrix} b_1 \\ b_2 \\ \vdots \\ b_B \\ h \end{bmatrix}. \quad (3)$$

The matrices A_i encode local restrictions. The matrices L_i encode each block's contribution to the global coupling rows, such as demand balance in the hydrothermal testbed.

This structure suggests three scaling decisions: scale local rows inside each block; compute representative block scales after local row scaling; and scale global coupling rows with those block scales. The goal is to reduce harmful scale heterogeneity without erasing the modular meaning of the formulation.

3.3. Algebraic equivalence and spectral diagnostics

We restrict the preprocessing layer to positive row scalings. This restriction makes the transformation auditable: variables keep their original units, and the generated MIP remains the same optimization problem. Let $D_r = \text{diag}(d_1, \dots, d_m)$, with $d_r > 0$ for every row. We replace (A, b) by

$$\tilde{A} = D_r A, \quad \tilde{b} = D_r b. \quad (4)$$

For every vector x ,

$$Ax \leq b \iff D_r Ax \leq D_r b, \quad (5)$$

because each component of the inequality is multiplied by a strictly positive scalar. Thus positive row scaling preserves the feasible set, the integer restrictions, and the objective function. Any difference observed after preprocessing must therefore come from the numerical representation seen by the solver, not from a different mathematical MIP.

The theoretical arguments below use the spectral condition number as a diagnostic of linear-algebraic sensitivity. For a real matrix M , define

$$\kappa_2(M) = \frac{\sigma_{\max}(M)}{\sigma_{\min}^+(M)}, \quad (6)$$

where $\sigma_{\max}(M)$ is the largest singular value and $\sigma_{\min}^+(M)$ is the smallest positive singular value. For rectangular matrices, this ratio is used only as a diagnostic over the nonzero singular

spectrum; in square nonsingular benchmarks, rank deficiency is interpreted as infinite condition number. This diagnostic does not characterize the full difficulty of a MIP. A MIP solver also depends on presolve, cuts, branching, primal heuristics, incumbent discovery, feasibility tolerances, and integrality. The role of κ_2 is narrower: it provides an idealized matrix argument explaining why local rows and coupling rows should not be scaled in exactly the same way.

3.4. Ideal block scaling

The block structure is not only a modeling convenience. It identifies what scalar scaling can and cannot repair. In a purely block-diagonal matrix $D = \text{diag}(A_1, \dots, A_B)$, a single scalar per block can remove differences of scale between blocks, but it cannot improve the intrinsic conditioning of an individual block. The following benchmark makes this statement precise in the ideal square nonsingular case.

Proposition 1 (Ideal scalar scaling of block-diagonal matrices). *Let $D = \text{diag}(A_1, \dots, A_B)$, where every A_i is square and nonsingular. Let $M_i = \sigma_{\max}(A_i)$ and $m_i = \sigma_{\min}(A_i) > 0$. Then*

$$\min_{\alpha_i > 0} \kappa_2(\text{diag}(\alpha_1 I, \dots, \alpha_B I) D) = \max_i \kappa_2(A_i). \quad (7)$$

One minimizer is

$$\alpha_i = \frac{1}{\sqrt{M_i m_i}} = \frac{1}{\sqrt{\sigma_{\max}(A_i) \sigma_{\min}(A_i)}}. \quad (8)$$

Proof. For fixed positive scalars α_i , the scaled matrix is block diagonal:

$$\text{diag}(\alpha_1 I, \dots, \alpha_B I) D = \text{diag}(\alpha_1 A_1, \dots, \alpha_B A_B).$$

The singular values of a block-diagonal matrix are the union of the singular values of its blocks. Therefore, for every i ,

$$\kappa_2(\text{diag}(\alpha_1 A_1, \dots, \alpha_B A_B)) \geq \kappa_2(\alpha_i A_i) = \kappa_2(A_i).$$

Taking the maximum over i gives the lower bound

$$\kappa_2(\text{diag}(\alpha_1 A_1, \dots, \alpha_B A_B)) \geq \max_i \kappa_2(A_i)$$

for every admissible block scaling.

Now choose $\alpha_i = (M_i m_i)^{-1/2}$. Then the largest and smallest singular values of $\alpha_i A_i$ are

$$\begin{aligned} \sigma_{\max}(\alpha_i A_i) &= \sqrt{\frac{M_i}{m_i}} = \sqrt{\kappa_2(A_i)}, \\ \sigma_{\min}(\alpha_i A_i) &= \sqrt{\frac{m_i}{M_i}} = \frac{1}{\sqrt{\kappa_2(A_i)}}. \end{aligned}$$

Because the singular values of the complete block-diagonal matrix are obtained by pooling the singular values of its blocks,

$$\sigma_{\max}(\text{diag}(\alpha_1 A_1, \dots, \alpha_B A_B)) = \sqrt{\max_i \kappa_2(A_i)},$$

and

$$\sigma_{\min}(\text{diag}(\alpha_1 A_1, \dots, \alpha_B A_B)) = \frac{1}{\sqrt{\max_i \kappa_2(A_i)}}.$$

The resulting condition number is $\max_i \kappa_2(A_i)$, matching the lower bound. $\square$

This proposition is not used as an algorithmic recipe. The generated MIPs considered in this paper contain rectangular local matrices, inequalities, bounds, redundant rows, and integer variables; computing singular-value-based block scalings would not be a practical external preprocessing rule for the exported MIP. The proposition instead justifies the architecture of the method. First, poor scale separation between blocks should be treated differently from poor conditioning inside a block. Second, after local row scaling, each block should contribute a representative scale before global coupling rows are processed. The implemented rules therefore replace the ideal singular-value quantities (M_i, m_i) with coefficient-based proxies that are cheap to compute from the generated model. These propositions are design principles, not performance theorems. They should not be read as predictive guarantees for the implemented MIP preprocessing rule; they motivate the distinction between local rows, block scales, and coupling rows.

3.5. Effect of global coupling rows

Local block scaling does not exhaust the numerical issue because the generated model is not purely block diagonal. In the hydrothermal dispatch testbed, global rows couple local agent decisions through demand and balance equations. The relevant schematic block is

$$A_B = \begin{bmatrix} D & 0 \\ L & -I_T \end{bmatrix}, \quad (9)$$

where $D = \text{diag}(A_1, \dots, A_B)$ represents the local blocks and L collects the coefficients by which local decisions enter the global balance. If D is nonsingular, then

$$A_B = \begin{bmatrix} I & 0 \\ LD^{-1} & -I_T \end{bmatrix} \begin{bmatrix} D & 0 \\ 0 & I_T \end{bmatrix}. \quad (10)$$

This factorization separates local scale from coupling scale. The matrix L by itself is not the relevant object: the coupling felt by the balance row after the local block scales are taken into account is LD^{-1} . Thus a row with moderate entries in L can still be numerically influential if some local block contains weak directions.

The next result makes the dependence on the effective coupling norm explicit.

Proposition 2 (Conditioning of a triangular coupling factor). *Let $R \in \mathbb{R}^{m \times n}$ and define*

$$T_R = \begin{bmatrix} I_n & 0 \\ R & I_m \end{bmatrix}. \quad (11)$$

If $\rho = \|R\|_2$, then

$$\kappa_2(T_R) = \left(\frac{\sqrt{\rho^2 + 4} + \rho}{2} \right)^2. \quad (12)$$

Proof. Let $R = U \Sigma V^T$ be a singular value decomposition of R , with U and V orthogonal. Orthogonal multiplication does

not change singular values, and hence T_R has the same singular values as

$$\begin{bmatrix} V^\top & 0 \\ 0 & U^\top \end{bmatrix} \begin{bmatrix} I_n & 0 \\ R & I_m \end{bmatrix} \begin{bmatrix} V & 0 \\ 0 & U \end{bmatrix} = \begin{bmatrix} I_n & 0 \\ \Sigma & I_m \end{bmatrix}.$$

For each singular value σ of R , the nontrivial part reduces to the two-dimensional block

$$B_\sigma = \begin{bmatrix} 1 & 0 \\ \sigma & 1 \end{bmatrix}.$$

The squared singular values of B_σ are the eigenvalues of

$$B_\sigma^\top B_\sigma = \begin{bmatrix} 1 + \sigma^2 & \sigma \\ \sigma & 1 \end{bmatrix},$$

namely

$$\lambda_\pm(\sigma) = \frac{\sigma^2 + 2 \pm \sigma \sqrt{\sigma^2 + 4}}{2}.$$

Their product is one, and $\lambda_+(\sigma)$ is increasing for $\sigma \geq 0$. Additional directions associated with zero singular values of R contribute singular values equal to one. Therefore the largest squared singular value of T_R is $\lambda_+(\rho)$ and the smallest squared singular value is $1/\lambda_+(\rho)$. Consequently,

$$\begin{aligned} \kappa_2(T_R) &= \lambda_+(\rho) \\ &= \frac{\rho^2 + 2 + \rho \sqrt{\rho^2 + 4}}{2} \\ &= \left(\frac{\sqrt{\rho^2 + 4} + \rho}{2} \right)^2. \end{aligned}$$

□

Applied to (10), the proposition shows that the balance row should not be treated as an ordinary local row. Its numerical effect depends on the effective coupling proxy $\|LD^{-1}\|_2$, not only on the raw magnitude of the entries in L . Moreover, if a common positive factor γ is applied to the coupling row, the scaled block satisfies

$$\begin{bmatrix} D & 0 \\ \gamma L & -\gamma I_T \end{bmatrix} = \begin{bmatrix} I & 0 \\ \gamma LD^{-1} & -I_T \end{bmatrix} \begin{bmatrix} D & 0 \\ 0 & \gamma I_T \end{bmatrix}. \quad (13)$$

Thus γ changes both the effective coupling term and the relative scale of the aggregate variable in the balance equation. This is the trade-off behind the proposed coupling-row factor: global rows should be scaled after local rows, using block-level representative scales, rather than by an isolated row norm. In the implemented variants, D^{-1} and singular values are replaced by coefficient-based block proxies, yielding a cheap operational approximation to the ideal coupling measure.

3.6. From ideal diagnostics to implementable proxies

The two propositions above are not meant to prescribe an SVD-based scaling algorithm. Their role is to identify the numerical distinctions that an implementable generator-aware rule should preserve. In particular, they imply three design

requirements. First, local scale imbalance should be reduced before global coupling rows are diagnosed. Second, blocks should be summarized only after their local rows have been put on a comparable scale. Third, a coupling row should be scaled relative to the blocks it connects, not only relative to its own raw coefficient norm.

The implemented method replaces the ideal quantities by coefficient-based proxies for three practical reasons. The proxies are cheap to compute from the generated model before export; they remain auditable because they depend only on visible row and block metadata; and they avoid using expensive or unstable singular-value information from rectangular, redundant, inequality-based MIP matrices. The row factor based on the geometric mean of the smallest and largest nonzero magnitudes is therefore not intended as an approximation to the optimal spectral factor in [proposition 1](#). It is a robust coefficient-scale proxy that centers each eligible row without changing variables, bounds, integrality, or the objective.

The same interpretation applies to the coupling rule. In [proposition 2](#), the relevant ideal object is the effective coupling magnitude after local scaling, represented by a term of the form LD^{-1} . In generated MIPs, the local blocks are often rectangular, may contain redundant rows and bounds, and are not inverted by the preprocessing layer. The block scale s_i used below is therefore a representative coefficient scale for block i , computed after local row scaling. The coupling proxy divides each coupling coefficient by the representative scale of the block owning its column. This is a cheap operational surrogate for asking whether the coupling row is large relative to the local numerical scale of the blocks it links.

This bridge also explains why the proposed method is row-based. Positive row scaling preserves the original units and domains of all variables; integer and binary variables do not require reinterpretation, and objective coefficients are left in their original economic units. Column scaling can be useful as a generic numerical device, so a Ruiz-plus-column baseline is included in the experiments. It is not part of the proposed generator-aware layer because the central claim is that row-role and coupling metadata alone already provide useful pre-export numerical information while keeping the transformation algebraically transparent.

4. Numerical scaling method

The scaling method keeps the same structural roles visible throughout: local rows, blocks, and coupling rows. Local rows receive row factors first. Blocks then inherit representative scales from their scaled local rows. Coupling rows receive their factors last, using the blocks they connect. This order is the algorithmic expression of the numerical argument in [sections 3.4](#) and [3.5](#) and of the proxy interpretation in [section 3.6](#).

This section defines the numerical transformations tested in the results. The base variant is the unscaled model generated by the platform. The two proposed variants use the same local-block-coupling architecture but differ in whether the right-hand side participates in the scale diagnosis. Ruiz-type variants provide generic flat-matrix baselines. This organization makes

two contrasts explicit. The comparison between SA-Aug and SA-Mat isolates the role of right-hand-side information within the same architecture; the comparison with Ruiz-type baselines tests whether generator-level row roles add information beyond generic flat-matrix equilibration.

4.1. Base formulation

The Base variant is passed to the solver without external scaling. Internal solver presolve and internal scaling remain enabled unless stated otherwise. All speedups, objective deviations, and equivalence checks are computed relative to this variant on the same instance, solver, and gap setting.

4.2. Augmented generator-aware scaling

The first proposed variant, denoted SA-Aug, computes row factors from both coefficient magnitudes and right-hand sides. The motivation is that a generated row may have moderate coefficients but a right-hand side in a very different physical or economic scale. Such a row can still lead to a poorly balanced numerical representation.

For each local row r , define

$$\mathcal{S}_r^{A,b} = \{|a_{rj}| : a_{rj} \neq 0\} \cup \{|b_r| : b_r \neq 0\}. \quad (14)$$

If $\mathcal{S}_r^{A,b} \neq \emptyset$, let

$$M_r^{A,b} = \max \mathcal{S}_r^{A,b}, \quad m_r^{A,b} = \min \mathcal{S}_r^{A,b}. \quad (15)$$

The local row factor is

$$\alpha_r^{A,b} = \frac{1}{\sqrt{M_r^{A,b} m_r^{A,b}}}. \quad (16)$$

The scaled row is

$$\alpha_r^{A,b} a_r^\top x \leq \alpha_r^{A,b} b_r. \quad (17)$$

The augmented rule is not only a row-by-row normalization. It is a three-stage generator-aware procedure that first rescales local rows, then summarizes the resulting local block scales, and finally uses those block scales to treat coupling rows separately. After local scaling, let $\tilde{a}_{rj} = \alpha_r^{A,b} a_{rj}$ for local rows. For each local block i , define

$$M_i^{A,b} = \max_{r \in \mathcal{R}_i, j: \tilde{a}_{rj} \neq 0} |\tilde{a}_{rj}|, \quad m_i^{A,b} = \min_{r \in \mathcal{R}_i, j: \tilde{a}_{rj} \neq 0} |\tilde{a}_{rj}|. \quad (18)$$

The representative scale of block i is

$$s_i^{A,b} = \sqrt{M_i^{A,b} m_i^{A,b}}. \quad (19)$$

For a coupling row r , let $b(j)$ denote the local block associated with column j . The effective coupling proxy used by SA-Aug is

$$\widehat{\rho}_r^{A,b} = \sqrt{\sum_j \left(\frac{|a_{rj}|}{s_{b(j)}^{A,b}} \right)^2}. \quad (20)$$

If $\widehat{\rho}_r^{A,b} > 0$, the coupling-row factor is

$$\gamma_r^{A,b} = \frac{1}{\widehat{\rho}_r^{A,b}}. \quad (21)$$

All factors are then clipped by the safeguards described below. This sequence is the main structural feature of the proposed method: local physical scales are used first, block scales are estimated after local normalization, and global coupling rows are scaled only after the blocks they connect have been put on comparable numerical footing.

4.3. Matrix-only generator-aware scaling

The second proposed variant, denoted SA-Mat, uses the same row-block-coupling architecture but excludes b from the diagnosis. The right-hand side is still multiplied by the selected row factor to preserve equivalence, but it does not influence factor selection.

For each local row r , define the nonzero coefficient extremes

$$M_r^A = \max_{j: a_{rj} \neq 0} |a_{rj}|, \quad m_r^A = \min_{j: a_{rj} \neq 0} |a_{rj}|. \quad (22)$$

If the row has at least one nonzero coefficient, the row factor is

$$\alpha_r^A = \frac{1}{\sqrt{M_r^A m_r^A}}. \quad (23)$$

The scaled row is

$$\alpha_r^A a_r^\top x \leq \alpha_r^A b_r. \quad (24)$$

This factor centers the smallest and largest nonzero coefficient magnitudes of the row around a common scale. Unlike Euclidean-norm scaling, it does not directly depend on the number of nonzeros in the row.

Let $\tilde{a}_{rj} = \alpha_r^A a_{rj}$ after local row scaling. For each local block i , define

$$M_i^A = \max_{r \in \mathcal{R}_i, j: \tilde{a}_{rj} \neq 0} |\tilde{a}_{rj}|, \quad m_i^A = \min_{r \in \mathcal{R}_i, j: \tilde{a}_{rj} \neq 0} |\tilde{a}_{rj}|. \quad (25)$$

The representative block scale is

$$s_i^A = \sqrt{M_i^A m_i^A}. \quad (26)$$

No additional multiplier is applied to all rows or columns of block i at this stage. The quantity s_i^A is an auxiliary representative scale: it is used only to normalize the contribution of block i when the coupling-row proxy is computed.

For a coupling row r , let $b(j)$ denote the block associated with column j . Define

$$\widehat{\rho}_r^A = \sqrt{\sum_j \left(\frac{|a_{rj}|}{s_{b(j)}^A} \right)^2}. \quad (27)$$

If $\widehat{\rho}_r^A > 0$, the coupling-row factor is

$$\gamma_r^A = \frac{1}{\widehat{\rho}_r^A}. \quad (28)$$

The comparison between SA-Aug and SA-Mat isolates the empirical value of including the right-hand side in the scaling diagnosis. Both variants preserve the same structural architecture; they differ only in whether b participates in factor selection.

4.4. Safeguards

All applied scaling factors—the local row factors and the coupling-row factors in the augmented or matrix-only variants—are subject to two per-factor safeguards, applied independently to each factor. The representative block scales s_i are auxiliary quantities used to compute coupling-row proxies; they are not applied as independent row or column scalings.

Near-identity threshold. A factor is applied only if it departs from one by at least a factor of $u = 2$; factors falling in the band $[1/u, u] = [0.5, 2]$ are treated as negligible and skipped:

$$\theta \text{ is applied} \iff \left(\theta \leq \frac{1}{u} \right) \text{ or } (\theta \geq u), \quad u = 2. \quad (29)$$

This prevents re-scaling rows or coupling rows that are already at a reasonable scale.

Admissible coefficient range. A factor is applied only if it keeps every affected coefficient within the window $[\varepsilon_{\min}, \varepsilon_{\max}] = [10^{-9}, 10^9]$. Writing $c_{\min}$ and $c_{\max}$ for the smallest and largest magnitudes of the affected coefficients, the factor θ is admitted only if

$$\begin{aligned} \varepsilon_{\min} &\leq c_{\min} \theta, & c_{\max} \theta &\leq \varepsilon_{\max}, \\ [\varepsilon_{\min}, \varepsilon_{\max}] &= [10^{-9}, 10^9]. \end{aligned} \quad (30)$$

Otherwise, the factor is not applied and the corresponding row or coupling row is left at its base scale. The coupling factor is additionally hard-clipped to $\gamma \in [10^{-8}, 10^8]$ prior to this check.

Because each admitted factor is a strictly positive multiplier applied to a complete constraint—both its coefficients and its right-hand side—the feasible set and the objective value are preserved exactly. The safeguards therefore restrict only the numerical aggressiveness of the transformation, never its equivalence to the base model.

4.5. Ruiz-type baselines

We compare the proposed variants with two generic baselines. The first, Ruiz, applies iterative row equilibration based on the maximum norm. The second, Ruiz+Cols, combines Ruiz-type row equilibration with continuous-column scaling and physical normalization. These baselines separate the effect of generic coefficient compression from the effect of exploiting model structure.

The final implementation uses

$$K = 4 \quad (31)$$

Ruiz iterations. Binary and integer variables are not rescaled unless explicitly stated, so that bounds, objective interpretation, and integrality remain unchanged.

4.6. Algorithmic summary

Algorithm 1 Augmented generator-aware row scaling

Require: Matrix A , right-hand side b , block partition $\mathcal{B}$
Ensure: Scaled matrix $\tilde{A}$, scaled right-hand side $\tilde{b}$

- 1: Initialize row factors $d_r \leftarrow 1$
- 2: **for** each local block $i \in \mathcal{B}$ **do**
- 3: **for** each local row $r \in \mathcal{R}_i$ **do**
- 4: Build $\mathcal{S}_r^{A,b}$ from nonzero coefficients and nonzero right-hand side
- 5: **if** $\mathcal{S}_r^{A,b} \neq \emptyset$ **then**
- 6: Compute $M_r^{A,b}$ and $m_r^{A,b}$ using [equation \(15\)](#)
- 7: Set $\alpha_r^{A,b} \leftarrow 1/\sqrt{M_r^{A,b} m_r^{A,b}}$
- 8: Apply safeguards and update $d_r \leftarrow d_r \alpha_r^{A,b}$
- 9: **end if**
- 10: **end for**
- 11: **end for**
- 12: Compute the locally scaled coefficients $\tilde{a}_{rj} = d_r a_{rj}$
- 13: Compute block scales $s_i^{A,b} = \sqrt{M_i^{A,b} m_i^{A,b}}$ using [equation \(19\)](#)
- 14: **for** each coupling row r **do**
- 15: Compute $\hat{\rho}_r^{A,b}$ using [equation \(20\)](#)
- 16: **if** $\hat{\rho}_r^{A,b} > 0$ **then**
- 17: Set $\gamma_r^{A,b} \leftarrow 1/\hat{\rho}_r^{A,b}$
- 18: Apply safeguards and update $d_r \leftarrow d_r \gamma_r^{A,b}$
- 19: **end if**
- 20: **end for**
- 21: Return $\tilde{A} = \text{diag}(d)A$ and $\tilde{b} = \text{diag}(d)b$

The matrix-only algorithm is obtained from [algorithm 1](#) by replacing $\mathcal{S}_r^{A,b}$ with the coefficient-only set $\{a_{rj} : a_{rj} \neq 0\}$ and by using the representative block scales and coupling proxies in [equations \(26\)](#) and [\(27\)](#). In the present manuscript, SA-Mat is interpreted as an ablation of the right-hand-side information: it preserves the same local-row and coupling-row architecture, but the right-hand side does not enter the factor-selection rule.

5. Experimental design

5.1. Instance classes and scenarios

The experiments are organized around four reader questions. Does scaling preserve the original optimization problem? Does it improve the numerical representation? Does that improvement change solver behavior? Does the effect survive changes in instance class, solver, and tolerance? To answer these questions, the computational study uses three instance classes generated by the same modular hydrothermal dispatch platform. The pools are built from operational instances for which the required real input data are available. Consequently, the number of valid runs may differ across class–scenario–variant combinations: some instances may be absent because the corresponding real data, generated model, or solver log is unavailable. Descriptive aggregate metrics use all valid runs for the reported row. Speedups, statistical tests, and all claims involving direct comparison with

Table 1: The experimental design turns the main claim into reader questions, evidence, and possible objections.

Reader question	Evidence reported in the paper	Objection addressed by the design
Does scaling change the optimization problem?	Algebraic equivalence of positive row scaling, descaling checks, and original-scale feasibility/integrity tests.	A faster run would be irrelevant if it solved a different MIP or hid violations created by rescaling.
Does generator awareness improve numerical representation?	Coefficient-range summaries, estimated conditioning ratios, and solver-level conditioning diagnostics when available.	Static matrix compression alone may not measure the numerical state actually used by the solver.
Does numerical improvement become algorithmic improvement?	Matched base-variant speedups, deterministic effort, performance profiles, and primal-dual progress.	Better conditioning diagnostics do not necessarily imply shorter branch-and-bound search.
Is the effect a property of the method or of one solver pipeline?	Separate Gurobi and CPLEX scenarios at two MIPGap tolerances.	A positive result under one solver may be absorbed, reversed, or made irrelevant by another solver’s internal preprocessing and search strategy.

The table is used as a guide to the results. It separates claims from the evidence that can support them and keeps the strongest performance claim conditional on solver behavior.

Table 2: Three instance classes test the method on progressively different generated hydrothermal models.

Class	Configuration	Main role	Instances
Simple	Río Negro + fixed demand + fixed dispatch	Basic numerical behavior	204
Full	Río Negro nonlinear + Salto Grande + thermal units + market	Largest integrated model	34
SG-Ter-Mer	Salto Grande + thermal units + market	Coupled hydrothermal subset	68

The counts report the maximum operational instance pools with available real input data. Scenario tables may show different n values because each aggregate row uses the valid runs available for that solver-gap-variant combination. Pairwise speedups and tests use matched base-variant pairs only.

the base model use matched base-variant pairs for the same instance, solver, and gap tolerance.

Table 1 states the logic of the computational study. The experiments are designed not only to report faster or slower runs, but also to show which part of the argument each measurement can support.

Table 2 separates the three experimental pools before solver results are compared. Simple gives the largest sample for distributional evidence, Full is the smallest and most integrated model, and SG-Ter-Mer isolates a coupled hydrothermal subset. This distinction matters because the descriptive averages and paired tests do not come from a single homogeneous instance population.

The class names are application-specific, but the experimental contrast is structural. The study asks whether the preprocessing rule behaves consistently as the generated MIP moves from a simpler modular model to larger or more strongly coupled block-structured models. The conclusions below are therefore stated at two levels: the empirical evidence is from hydrothermal dispatch, while the methodological object is the use of generator metadata in block-structured engineering MIPs.

We evaluate four solver-gap scenarios. The time limit is one hour per solve in every case. Internal solver presolve and internal scaling are left enabled, because the goal is to test whether external preprocessing helps in a realistic solver pipeline. All main experiments were run on ClusterUY, a collaborative high-performance computing platform for scientific computing in Uruguay [11].

The four scenarios vary two dimensions separately: the stopping tolerance and the solver pipeline. Scenarios 1–2 test Gurobi at two MIPGap values; scenarios 3–4 repeat the comparison under

CPLEX and serve as a solver contrast, not as another route to a universal acceleration claim.

This configuration matters because the proposed preprocessing is tested on top of each solver’s default numerical pipeline. The CPLEX conclusions below are therefore statements about this version and ClusterUY environment, not claims about all CPLEX releases or parameter choices.

5.2. Controlled synthetic block-structured benchmark

The hydrothermal testbed evaluates the proposed preprocessing rule on realistic operational generated MIPs. To complement this real-data setting, we define a controlled synthetic benchmark whose purpose is diagnostic rather than physical. The synthetic instances are not intended to model a power system. They isolate the numerical mechanism targeted by the method by allowing local-row scale imbalance and coupling-row scale imbalance to be varied independently.

Each synthetic instance consists of B local blocks. Block i contains continuous activity variables $y_i \in \mathbb{R}_+^{n_y}$ and binary activation variables $z_i \in \{0, 1\}^{n_z}$. A nonnegative slack vector s is added to the global coupling constraints to guarantee feasibility. The generic synthetic MIP is

$$\begin{aligned}
\min_{y, z, s} \quad & \sum_{i=1}^B c_i^\top y_i + \sum_{i=1}^B q_i^\top z_i + P \mathbf{1}^\top s \\
\text{s.t.} \quad & R_i y_i + G_i z_i \leq r_i, \quad i = 1, \dots, B, \\
& 0 \leq y_{ik} \leq U_{ik} z_{ik}, \quad i = 1, \dots, B, \quad k = 1, \dots, n_y, \quad (32) \\
& \sum_{i=1}^B L_i y_i + \sum_{i=1}^B H_i z_i + s \geq d, \\
& s \geq 0, \quad z_i \in \{0, 1\}^{n_z}.
\end{aligned}$$

Table 3: Four solver–tolerance scenarios separate solver dependence from gap-tolerance effects.

Scenario	Solver	Relative MIPGap
1	Gurobi	10^{-2}
2	Gurobi	10^{-3}
3	CPLEX	10^{-2}
4	CPLEX	10^{-3}

Deterministic time is reported when available: `Work` units for Gurobi and ticks for CPLEX.

Table 4: The computational environment fixes solver versions, time limits, and default presolve settings for all comparisons.

Item	Configuration
Cluster	ClusterUY, partition <code>ute</code>
Processor	Intel Xeon Gold 6138
Threads	16
Memory limit	12 GB
Time limit	1 hour per solve
Gurobi version	13.0.1
CPLEX version	22.1.2.0
Presolve	solver default, enabled

Solver-internal scaling and presolve were left at their default enabled settings unless otherwise stated. CPLEX 22.1.2.0 is the version available in the ClusterUY environment used for the long runs; conclusions involving CPLEX should therefore be interpreted as solver-version and environment specific.

The first group of constraints defines local block restrictions, the second group links continuous activity variables to binary activation variables, and the third group creates global coupling rows. The penalty P is chosen large enough that the slack variable is used only when the activated block capacity cannot satisfy the synthetic demand.

The unscaled coefficients are generated from bounded random distributions with controlled sparsity. Local matrices R_i and G_i define block-internal resource constraints. Coupling matrices L_i and H_i define the contribution of each block to the global rows. The generator records row metadata

$$\text{row_type} \in \{\text{local}, \text{coupling}\}, \quad \text{block_id} \in \{1, \dots, B\}, \quad (33)$$

and column metadata identifying the block associated with each variable. This metadata is the same information required by the generator-aware scaling rules.

Scale imbalance is introduced after the clean synthetic MIP has been generated. For each eligible row r , a positive row multiplier

$$\theta_r = 10^{\xi_r}, \quad \xi_r \sim \text{Unif}[-S, S], \quad (34)$$

is drawn and applied to the entire row, including its right-hand side. Therefore, the scaled representation is algebraically equivalent to the clean synthetic MIP. The severity parameter S controls the possible scale range. We use

$$S \in \{0, 3, 6\}, \quad (35)$$

corresponding to no artificial imbalance, moderate imbalance, and strong imbalance.

Four scale patterns are considered:

1. **None:** no artificial row scaling is applied.
2. **Local:** only local block rows are multiplied by factors (34).
3. **Coupling:** only global coupling rows are multiplied by factors (34).
4. **Mixed:** both local block rows and global coupling rows are multiplied.

The “none” pattern is a negative control: a useful preprocessing rule should not create large artificial changes when the representation is already reasonably scaled. The “local” and “coupling” patterns isolate the two numerical objects distinguished by the proposed method. The “mixed” pattern approximates the combined heterogeneity observed in generated engineering models. These controlled patterns are mechanism checks. They verify that the implementation reacts correctly when the source of imbalance is known by construction, but they are not an operational local-only versus coupling-only ablation on the real hydrothermal instances.

The reported synthetic runs use the five scaling variants in section 5.3, the same original-scale feasibility and objective checks, Gurobi with relative MIPGap 10^{-3} , and a 600 second time limit. They are intentionally small: the benchmark is a controlled mechanism check for numerical conditioning and algebraic equivalence, not a performance experiment. Runtime claims are reserved for the operational hydrothermal instances.

Table 5: Controlled synthetic benchmark factors isolate the numerical mechanisms targeted by the scaling rule.

Factor	Levels used	Controlled object	Purpose
Scale pattern	none, local, coupling, mixed	Row class affected by artificial scaling	Separate local imbalance from coupling imbalance
Severity S	0, 3, 6	Range of row multipliers $10^{-S} - 10^S$	Test mild and severe numerical heterogeneity
Blocks and size	$B = 10, n_y = n_z = 8$	Number and size of generated local components	Keep a fixed block structure while varying row-scale imbalance
Rows and seeds	8 local rows per block, 3 coupling rows, 20 seeds	Random coefficients and sparsity patterns	Estimate distributional behavior rather than one instance

The synthetic instances are not physical power-system models. They are controlled block-structured MIPs designed to test whether the preprocessing rule responds to the type of scale imbalance it is designed to exploit. The reported synthetic grid uses the middle size $B = 10$, the four patterns, the three severity levels, and twenty random seeds per pattern–severity cell.

5.3. Scaling variants

The compared variants are: Base, the original formulation; SA-Aug, the augmented generator-aware scaling rule; SA-Mat, the matrix-only generator-aware rule with safeguards; Ruiz, generic Ruiz-type row equilibration with $K = 4$ iterations; and Ruiz+Cols, Ruiz-type row scaling plus continuous-column scaling and physical normalization. Binary and integer variables are not rescaled.

5.4. Metrics and reporting conventions

The analysis reports three families of metrics. First, feasibility preservation is checked after descaling the solver solution and evaluating it on the original MIP. The primary post-solve check is the maximum violation of the original constraints, together with the maximum integrality violation. A run is labeled feasible in original scale if the maximum row violation is at most 10^{-4} and the maximum integer violation is at most 10^{-5} . Recomputing the objective in original units is retained as an auxiliary consistency diagnostic, but it is not the primary feasibility criterion. Objective agreement with the base is measured by

$$\Delta z_{\text{rel}} = \frac{|z_v - z_{\text{base}}|}{\max\{1, |z_{\text{base}}|\}}. \quad (36)$$

Second, numerical behavior is a primary outcome, not only an explanatory variable for speed. We distinguish three quantities. The first is the spectral condition number used in [section 3.3](#) to motivate the rule in an idealized linear-algebraic setting. The second is the exported coefficient-range ratio

$$\rho_\kappa(v) = \frac{(\max |a| / \min |a|)_v}{(\max |a| / \min |a|)_{\text{base}}}, \quad (37)$$

where the extrema are computed over nonzero coefficients in the generated matrix. This is a static proxy and should not be interpreted as the condition number of the MIP. The third is the solver-level ratio

$$\rho_\kappa^{\text{solver}}(v) = \frac{\kappa_{\text{solver}}(v)}{\kappa_{\text{solver}}(\text{base})}, \quad (38)$$

where κ_{solver} is obtained from the fixed-integer LP after the main MIP solve when available. For CPLEX we also record tree-level KappaStats: maximum sampled basis condition number, fractions of unstable and ill-conditioned bases, and a numerical

attention score. Solver numerical warnings parsed from logs are retained as a descriptive alert, not as a ranking criterion. Values below one indicate improvement relative to the base. The paper therefore separates the numerical effect of the preprocessing from the question of whether that numerical effect is converted into faster MIP search.

Third, efficiency is measured by wall-clock time, solver time, total time including preprocessing, deterministic time, node counts, iterations, final gap, and the primal–dual integral. For any time measure T_q , the paired speedup is

$$\text{speedup}_q(v) = \frac{T_{q,\text{base}}}{T_{q,v}}, \quad q \in \{\text{solver, total, det}\}. \quad (39)$$

Values above one indicate that variant v is faster than the base. Because solution times are highly skewed, we report shifted geometric means

$$\text{sgm}_s(\{x_i\}) = \exp\left(\frac{1}{n} \sum_i \log(x_i + s)\right) - s, \quad (40)$$

using $s = 1$ s for times and $s = 10$ for node counts. Timeouts are summarized by PAR10, where a timeout counts as ten times the time limit. The primal–dual integral is

$$\text{PI} = \frac{1}{T} \int_0^T \gamma(t) dt, \quad (41)$$

where $\gamma(t)$ is the reported optimality gap during the run.

5.5. Statistical analysis

The statistical analysis is paired by construction: every variant is compared with the base formulation on the same instance, solver, and MIPGap tolerance. Since MIP solution times are strongly skewed and include censored observations, solver-time comparisons use the Wilcoxon signed-rank test on matched, non-double-timeout pairs. Pairs in which both the base and the variant reach the time limit are treated as censored for this paired test; their global effect is still reflected in PAR10 and in the Dolan–Moré performance profiles. Alongside the test statistic W , we report the raw p -value, the Benjamini–Hochberg adjusted value q_{BH} , and Cliff’s δ as an effect size. With the convention used here, negative values of δ indicate that the

variant tends to be faster than the base, whereas positive values indicate deterioration.

To evaluate the connection between numerical and algorithmic effects, we first report conditioning ratios directly by class and variant, and then compute Spearman correlations between solver speedup and two conditioning ratios: the exported coefficient-range ratio ρ_k and the solver-level conditioning ratio ρ_k^{solver} . The central comparison is not merely whether either correlation is significant, but whether the solver-level diagnostic is more informative than the static coefficient-range proxy on the same matched observations. We therefore compare the two overlapping dependent correlations using the Williams–Steiger test, which also uses the correlation between ρ_k and ρ_k^{solver} . Where relevant, the numerical audit also checks partial rank correlations controlling for instance size, to ensure that the association is not only a proxy for larger or harder instances.

6. Computational results

The results follow the hierarchy of claims. We first check whether the solver-facing representation improves. We then ask whether Gurobi converts that improvement into lower runtime or deterministic effort. Finally, we use CPLEX as a contrast to test whether the performance effect is solver-independent.

This order is important because numerical improvement and algorithmic acceleration are related, but not identical, outcomes. The main pattern is clear but bounded. Under Gurobi, the generator-aware variants improve solver-facing conditioning and usually reduce time or deterministic effort. Under CPLEX 22.1.2.0, the same external scalings do not produce a robust runtime gain. The section therefore moves from the positive Gurobi evidence to the CPLEX contrast, with each table adding one layer of evidence: efficiency, conditioning, paired tests, and solver dependence.

A conditioning table alone would imply too much, because a better-scaled matrix does not guarantee easier branch-and-bound search. A timing table alone would imply too little, because it would hide the numerical mechanism. The numerical and algorithmic evidence must therefore be read together.

6.1. Scenario 1: Gurobi with 1% MIPGap

At the looser Gurobi tolerance, the evidence favors the generator-aware family. SA-Mat gives the lowest shifted geometric solver time in Simple and SG-Ter-Mer, and it improves deterministic effort in all three classes. SA-Aug remains the strongest descriptive rule in Full. The Ruiz-type baselines do not tell the same story: they may change coefficient ranges, but their runtime behavior is irregular.

Table 6 shows that the Gurobi gains are not confined to one summary statistic. SA-Mat gives the best shifted geometric time and PAR10 in Simple and SG-Ter-Mer, while SA-Aug is strongest in Full by shifted geometric time and deterministic speedup. The Ruiz-type baselines do not show the same class-consistent pattern.

6.2. Scenario 2: Gurobi with 0.1% MIPGap

The stricter Gurobi tolerance tests whether the first pattern survives a harder stopping criterion. The pattern largely persists. The generator-aware variants remain the most favorable options, whereas Ruiz-type variants remain unstable. SA-Mat again gives the best shifted geometric solver time in Simple and SG-Ter-Mer, while SA-Aug is stronger in Full. This result suggests that, in this testbed, much of the useful structural scale information is already present in the coefficient matrix.

At the stricter tolerance, table 7 preserves the same qualitative picture. SA-Mat remains the most attractive descriptive rule in Simple and SG-Ter-Mer, whereas SA-Aug is the strongest generator-aware rule in Full. The Gurobi evidence therefore does not hinge on the looser 1% stopping criterion.

Table 8 separates two numerical notions that are easy to conflate. The generator-aware variants reduce both the exported coefficient-range proxy and the solver-level ratio. Ruiz+Cols, by contrast, may compress the exported matrix while worsening the solver-level diagnostic in Full.

The same table also clarifies why the structured variants are not just generic equilibration in another form. Both SA-Aug and SA-Mat reduce the solver-level conditioning ratio well below one. The matrix-only rule compresses ρ_k essentially like the augmented rule and brings ρ_k^{solver} to 0.07–0.08 in Full and SG-Ter-Mer, whereas Ruiz+Cols may improve the static range without improving the solver-facing diagnostic.

The paired tests in table 9 support the descriptive Gurobi pattern. The signs are favorable for both generator-aware variants, with the strongest support in Simple and SG-Ter-Mer for SA-Mat and in the stricter Full case for SA-Aug. The effect sizes are moderate rather than dramatic; the evidence is a distributional shift, not a guarantee that every instance accelerates.

Table 10 adds the aggregate correlation check. The exported coefficient-range ratio is essentially uninformative about speedup, while the solver-level conditioning ratio has the expected negative association. This is the empirical basis for treating solver-facing diagnostics as more informative than exported-matrix summaries.

Together with tables 8 and 10, these results support a two-step interpretation: generator-aware scaling changes the numerical representation seen by Gurobi, and the solver-level conditioning ratio is much more informative about speedup than the raw coefficient-range ratio.

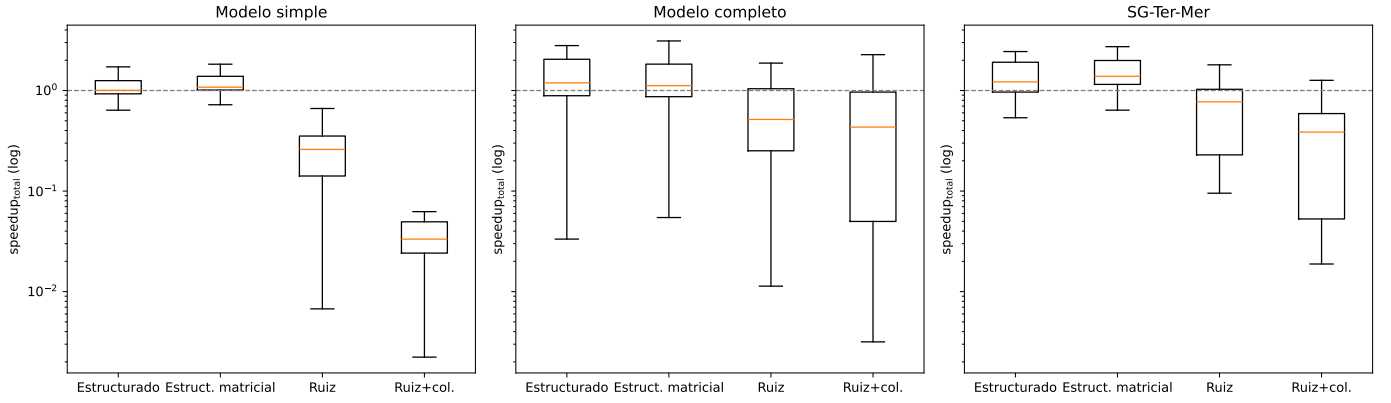


Figure 1: At 1% MIPGap, the Gurobi speedup distributions place the generator-aware variants mostly above parity, while Ruiz-type baselines are more dispersed. Values above the parity line indicate acceleration relative to the unscaled model.

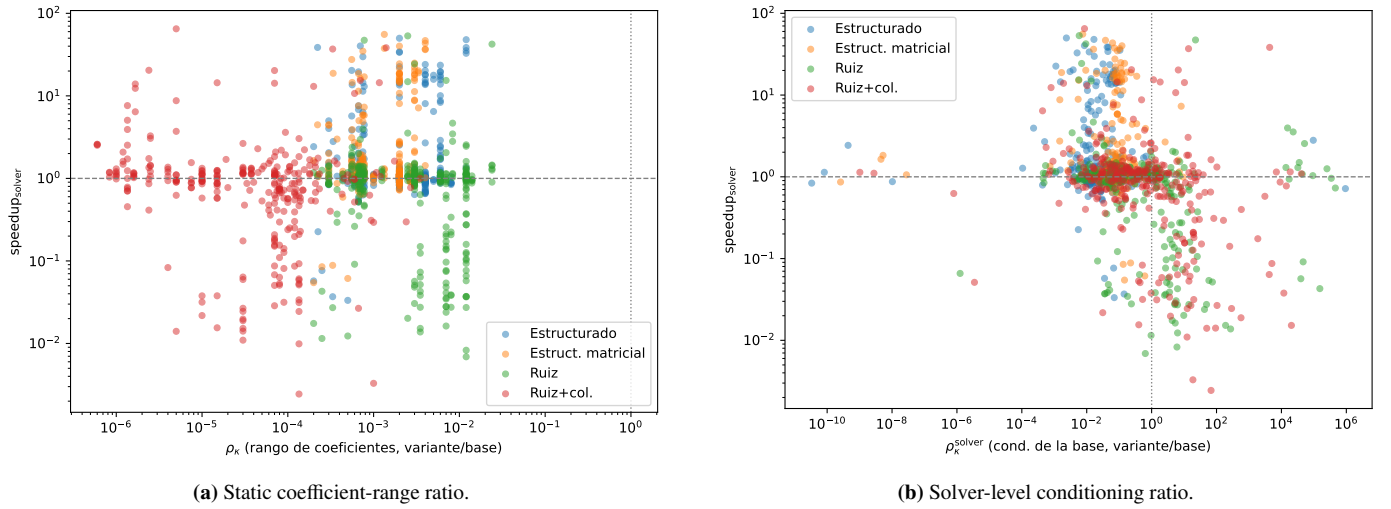


Figure 2: At 1% MIPGap, static coefficient-range compression and solver-level conditioning tell different stories. The contrast between panels shows why exported matrix compression alone should not be treated as evidence of easier branch-and-bound search.

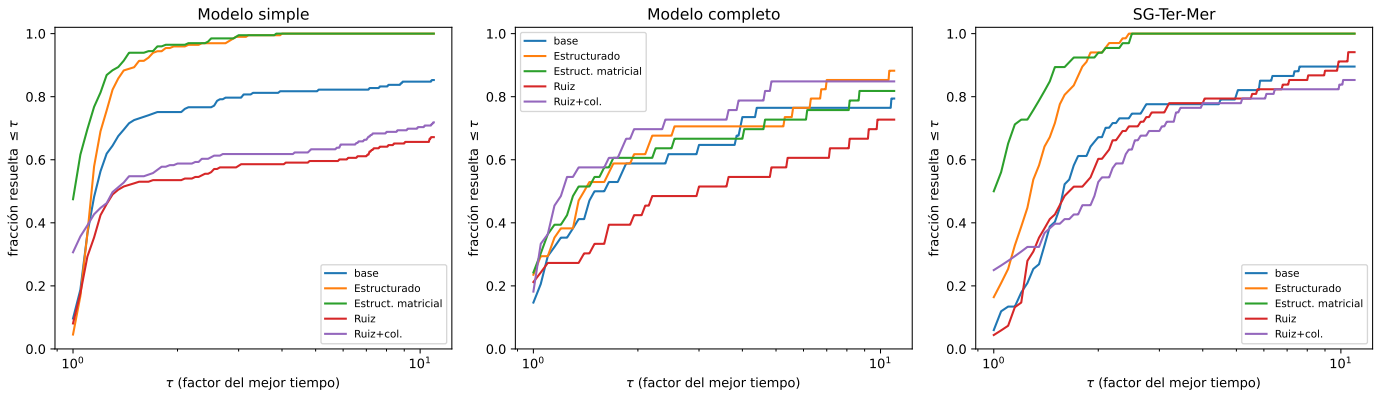


Figure 3: At 1% MIPGap, the Dolan–Moré profiles show whether the average Gurobi improvements persist across the instance pool. Curves farther toward the upper-left indicate variants that more often solve close to the best observed time.

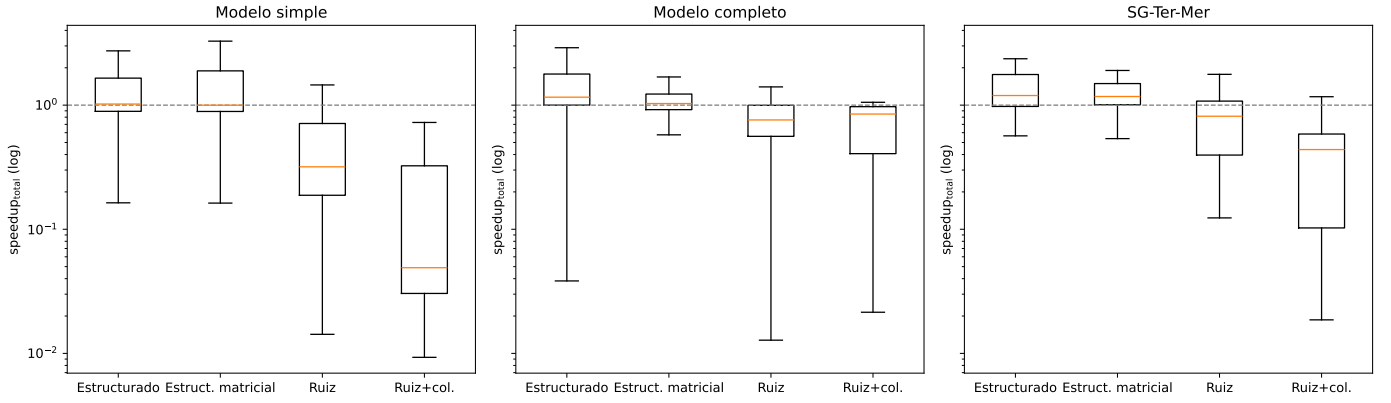


Figure 4: At 0.1% MIPGap, the Gurobi speedup distributions preserve the same qualitative pattern observed at the looser tolerance. Values above one indicate faster runs than the unscaled formulation.

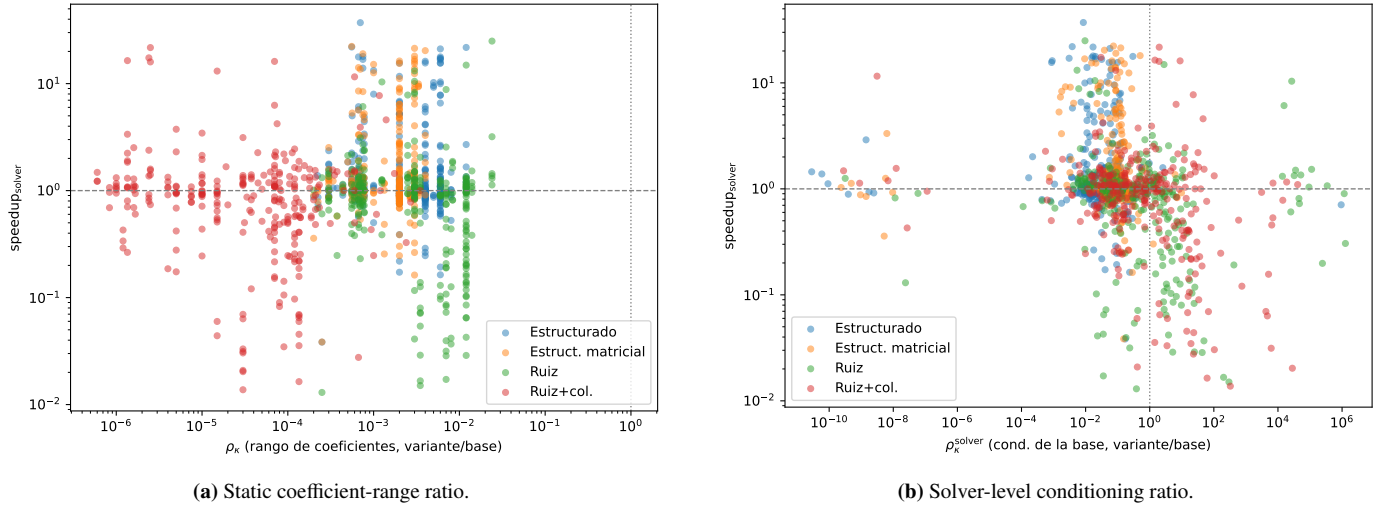


Figure 5: At 0.1% MIPGap, the stricter Gurobi runs again separate static coefficient compression from solver-facing numerical behavior. The solver-level diagnostic remains more informative than the exported coefficient-range ratio.

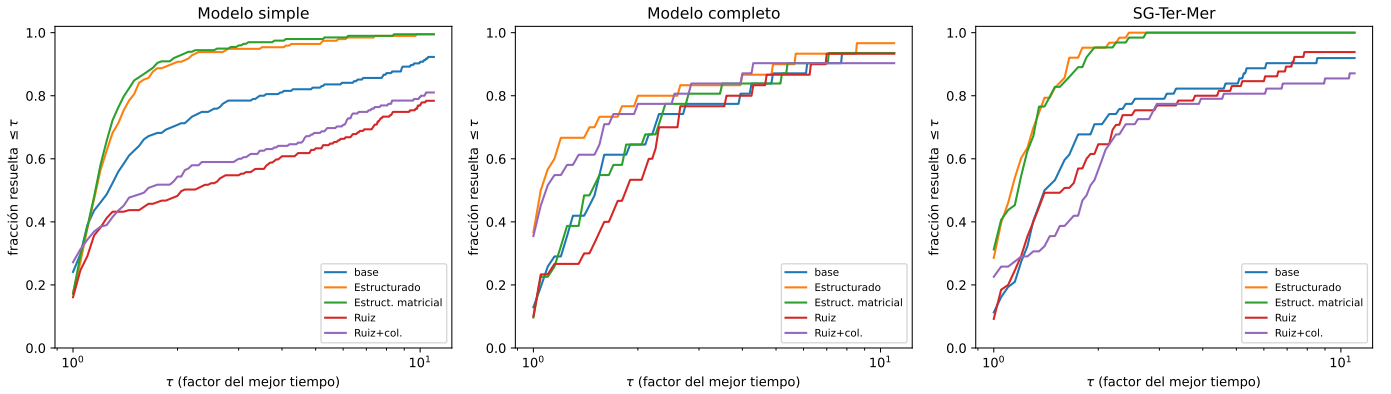


Figure 6: At 0.1% MIPGap, the Dolan–Moré profiles test whether the Gurobi speedups persist across the heterogeneous instance pool rather than only in averages.

Table 6: At 1% MIPGap, Gurobi converts generator-aware scaling into lower robust-efficiency metrics in the Simple and SG-Ter-Mer classes.

Class	Variant	n	sgm T	med. sp. ^{det}	PAR10	PI
Simple	Base	197	4.05	1.00	11.6	0.246
	SA-Aug	197	2.20	1.07	2.6	0.242
	SA-Mat	198	2.03	1.06	2.4	0.241
	Ruiz	198	8.27	0.88	26.7	0.300
	Ruiz+Cols	199	7.52	1.02	27.6	0.233
Full	Base	34	82.93	1.00	3384.8	0.120
	SA-Aug	34	70.56	1.37	3307.8	0.075
	SA-Mat	34	77.82	1.15	2487.2	0.055
	Ruiz	33	126.94	1.13	4566.5	0.263
	Ruiz+Cols	33	75.89	1.32	4696.3	0.239
SG-Ter-Mer	Base	67	14.14	1.00	572.7	0.664
	SA-Aug	67	8.39	1.32	562.2	0.566
	SA-Mat	67	6.75	1.40	21.0	0.531
	Ruiz	68	14.40	1.00	567.3	0.639
	Ruiz+Cols	68	18.35	0.78	585.1	0.662

Notes. sgm T : shifted geometric mean of solver time in seconds. med. sp.^{det}: median deterministic speedup with respect to the base model. PI: primal–dual integral. Lower is better for sgm T , PAR10, and PI; higher is better for med. sp.^{det}. Bold values mark the best descriptive value within each class and metric.

Table 7: At 0.1% MIPGap, the Gurobi efficiency pattern persists: generator-aware variants remain the strongest structured preprocessing rules.

Class	Variant	n	sgm T	med. sp. ^{det}	PAR10	PI
Simple	Base	195	8.28	1.00	28.5	0.176
	SA-Aug	195	5.65	1.11	28.7	0.181
	SA-Mat	198	5.44	1.10	21.1	0.182
	Ruiz	199	14.12	0.88	41.5	0.261
	Ruiz+Cols	195	12.55	1.02	48.9	0.199
Full	Base	31	305.30	1.00	5023.8	0.027
	SA-Aug	30	226.90	1.26	3890.2	0.028
	SA-Mat	31	306.39	1.12	5059.3	0.026
	Ruiz	30	341.56	0.87	5137.5	0.027
	Ruiz+Cols	31	267.10	1.62	6140.1	0.022
SG-Ter-Mer	Base	62	14.49	1.00	613.0	0.642
	SA-Aug	63	8.99	1.32	593.7	0.473
	SA-Mat	65	7.97	1.35	22.7	0.440
	Ruiz	65	14.27	1.05	586.8	0.634
	Ruiz+Cols	62	19.16	0.78	629.1	0.625

Notes. Bold values mark the best descriptive value within each class and metric; lower is better for sgm T , PAR10, and PI, whereas higher is better for med. sp.^{det}.

Table 8: Generator-aware scaling improves solver-facing conditioning diagnostics under Gurobi, whereas static coefficient compression alone is insufficient.

Class	Variant	n	median ρ_k (IQR)	median ρ_k^{solver} (IQR)	warnings
Simple	SA-Aug	195	0.00 (0.00)	0.04 (0.08)	1.0
	SA-Mat	195	0.00 (0.00)	0.08 (0.08)	1.0
	Ruiz	192	0.01 (0.01)	0.75 (3.94)	2.0
	Ruiz+Cols	190	1.5×10^{-5} (6.7×10^{-5})	1.19 (12.70)	1.0
Full	SA-Aug	30	5.6×10^{-4} (3.7×10^{-4})	0.02 (0.04)	1.0
	SA-Mat	31	5.6×10^{-4} (3.7×10^{-4})	0.07 (0.08)	1.0
	Ruiz	29	5.6×10^{-4} (3.7×10^{-4})	0.47 (2.55)	2.0
	Ruiz+Cols	30	6.0×10^{-4} (8.0×10^{-4})	1.60 (24.82)	0.0
SG-Ter-Mer	SA-Aug	62	6.7×10^{-4} (1.8×10^{-4})	0.01 (0.01)	1.0
	SA-Mat	61	6.7×10^{-4} (1.8×10^{-4})	0.08 (0.02)	1.0
	Ruiz	61	6.7×10^{-4} (1.8×10^{-4})	0.06 (0.06)	1.0
	Ruiz+Cols	59	1.0×10^{-4} (8.9×10^{-5})	0.07 (0.11)	0.0

Notes. Values below one indicate improvement relative to the base on the same scenario. ρ_k is the exported coefficient-range proxy, whereas ρ_k^{solver} is computed from the solver-level conditioning diagnostic on the fixed-integer LP. Entries shown as 0.00 are positive ratios below the displayed precision. The warnings column summarizes solver numerical warning flags parsed from logs and is used only as a descriptive diagnostic.

Table 9: Paired Gurobi tests show a distributional runtime shift in favor of the generator-aware variants, especially in Simple and SG-Ter-Mer.

Scenario	Class	Variant	W	p	q_{BH}	Cliff's δ
1% MIPGap	Simple	SA-Aug	7535.0	0.0057	0.0097	−0.090
1% MIPGap	Simple	SA-Mat	3011.0	$< 10^{-4}$	$< 10^{-4}$	−0.210
1% MIPGap	Full	SA-Aug	175.0	0.0984	0.1476	−0.082
1% MIPGap	Full	SA-Mat	172.0	0.1406	0.1875	−0.073
1% MIPGap	SG-Ter-Mer	SA-Aug	501.0	0.0001	0.0005	−0.138
1% MIPGap	SG-Ter-Mer	SA-Mat	115.0	$< 10^{-4}$	$< 10^{-4}$	−0.239
0.1% MIPGap	Simple	SA-Aug	6963.0	0.0010	0.0024	−0.077
0.1% MIPGap	Simple	SA-Mat	7069.0	0.0016	0.0028	−0.080
0.1% MIPGap	Full	SA-Aug	97.0	0.0262	0.0393	−0.144
0.1% MIPGap	Full	SA-Mat	173.0	0.7140	0.7140	−0.023
0.1% MIPGap	SG-Ter-Mer	SA-Aug	456.0	0.0004	0.0018	−0.145
0.1% MIPGap	SG-Ter-Mer	SA-Mat	341.0	$< 10^{-4}$	0.0002	−0.158

Notes. Tests are paired against the base formulation on matched instances. Negative Cliff's δ means that the variant tends to be faster. Double-timeout pairs are excluded from the Wilcoxon test and retained through PAR10 and the performance profiles.

Table 10: Solver-level conditioning is more informative about Gurobi speedup than the exported coefficient-range proxy.

Scenario	$\rho_S(\rho_k, s)$	p	$\rho_S(\rho_k^{\text{solver}}, s)$	p	Z	p_Z
1% MIPGap	+0.003	0.926	−0.256	3.7×10^{-19}	6.11	$< 10^{-4}$
0.1% MIPGap	+0.006	0.837	−0.267	5.6×10^{-20}	6.60	$< 10^{-4}$

Notes. s denotes solver speedup. The Williams–Steiger statistic Z compares the two overlapping correlations measured on the same matched observations. The negative solver-level correlations have the expected sign: lower solver-level conditioning ratios are associated with larger speedups.

6.3. Scenario 3: CPLEX with 1% MIPGap

The CPLEX scenarios provide a negative solver contrast to the Gurobi results. The structured variants remain close to parity and do not reproduce the systematic deterministic gains observed under Gurobi. Some descriptive wall-time improvements appear, especially in Full for SA-Mat, but the paired tests do not support robust generator-aware acceleration. In fact, some CPLEX signs are unfavorable for the structured variants. The correct interpretation is therefore not that CPLEX improves less than Gurobi, but that the present external scaling layer does not reliably improve CPLEX runtime in this environment. Ruiz is sometimes competitive in Full, but its performance is not stable across classes.

For CPLEX, [table 11](#) has to be interpreted differently from the Gurobi tables. Some descriptive summaries improve, especially for SA-Mat or Ruiz in Full, but the class-consistent generator-aware pattern is absent. In SG-Ter-Mer, the base scale already gives the best shifted geometric time and PAR10.

6.4. Scenario 4: CPLEX with 0.1% MIPGap

The stricter CPLEX setting confirms the same solver dependence and makes the negative runtime conclusion clearer. The structured variants remain at or near parity, the matrix-only rule essentially reproduces the base in deterministic effort, and the clearest descriptive improvement in shifted geometric solver time comes from Ruiz rather than from the proposed generator-aware rules. This is a useful negative result: external scaling may improve some numerical diagnostics without becoming a runtime accelerator for CPLEX in this environment.

The stricter CPLEX runs in [table 12](#) point in the same direction. Ruiz is descriptively best in several cells, while the two generator-aware variants remain close to the base. The result is solver dependence, not a weaker version of the Gurobi acceleration claim.

The paired CPLEX tests in [table 13](#) keep the descriptive tables in perspective. Unlike the Gurobi evidence, the signs are small, mixed, and sometimes unfavorable; after multiplicity adjustment they do not support a robust positive runtime effect for the generator-aware variants.

The summary table is mainly a roadmap. It makes the solver contrast visible at a glance: generator-aware variants dominate the Gurobi rows, whereas the CPLEX rows are heterogeneous and have to be interpreted together with the paired Wilcoxon tests.

6.5. Controlled synthetic mechanism check

The operational experiments above test the full solver–model interaction. The synthetic benchmark has a different role: it checks the numerical mechanism under controlled row-scale imbalance. The grid uses $B = 10$ blocks, each with 8 continuous and 8 binary variables, 8 local constraints plus binary-activation rows, and 3 global coupling rows with feasibility slacks. Row imbalance is injected through the pattern $\{none, local, coupling, mixed\}$ and $S \in \{0, 3, 6\}$. For each selected row r , all coefficients and the right-hand side are multiplied by $\theta_r = 10^{\xi_r}$, with $\xi_r \sim \mathcal{U}[-S, S]$. The grid contains 20 seeds per pattern–severity cell, for 240 instances and 1200 variant runs.

The design behaves as intended. For the negative-control pattern *none*, the median pre-scaling condition number is essentially invariant in S , around 1.3×10^3 . When imbalance is injected, κ^{pre} grows with severity only in the affected row classes: at $S = 6$, it reaches about 1.5×10^{14} for *local* and *mixed*, and about 4.9×10^8 for *coupling*. This confirms that the synthetic generator separates local-row and coupling-row imbalance by construction.

All scaling variants recover the conditioning despite the injected severity. In the severe local and mixed cases, κ^{pre} is of order 10^{14} , whereas the post-scaling medians fall between roughly 1.1×10^3 and 4.6×10^3 ([table 15](#)). The paired median reduction in $\log_{10} \kappa$ relative to the unscaled baseline reaches about eleven orders of magnitude for *local* and *mixed* at $S = 6$, and about 5.6 orders for *coupling* at $S = 6$. A paired Wilcoxon signed-rank analysis with Benjamini–Hochberg correction gives $p_{\text{BH}} \approx 1.9 \times 10^{-6}$ across the condition-number comparisons.

Table 11: At 1% MIPGap, CPLEX does not reproduce the Gurobi generator-aware acceleration pattern.

Class	Variant	n	sgm T	med. sp. ^{det}	PAR10	PI
Simple	Base	204	2.23	1.00	6.2	0
	SA-Aug	204	1.46	1.04	3.1	0
	SA-Mat	204	1.68	1.03	3.4	0
	Ruiz	204	1.49	0.99	3.6	0
	Ruiz+Cols	204	1.51	0.96	2.6	0
Full	Base	33	158.38	1.00	5621.7	3.1×10^{-8}
	SA-Aug	34	165.43	0.98	5546.0	3.2×10^{-8}
	SA-Mat	34	115.60	1.00	6497.8	2.3×10^{-8}
	Ruiz	34	115.96	1.14	3506.3	2.1×10^{-8}
	Ruiz+Cols	34	225.94	0.70	4681.1	5.5×10^{-8}
SG-Ter-Mer	Base	68	2.21	1.00	3.5	1.0×10^{-10}
	SA-Aug	68	2.37	1.02	9.9	1.0×10^{-10}
	SA-Mat	68	2.41	1.00	5.5	1.1×10^{-10}
	Ruiz	68	2.33	1.01	10.3	9.7×10^{-11}
	Ruiz+Cols	68	3.18	0.88	15.2	1.0×10^{-10}

Notes. Bold values mark the best descriptive value within each class and metric. Since the CPLEX statistical tests do not support a robust positive external-scaling effect, these bold entries are descriptive only.

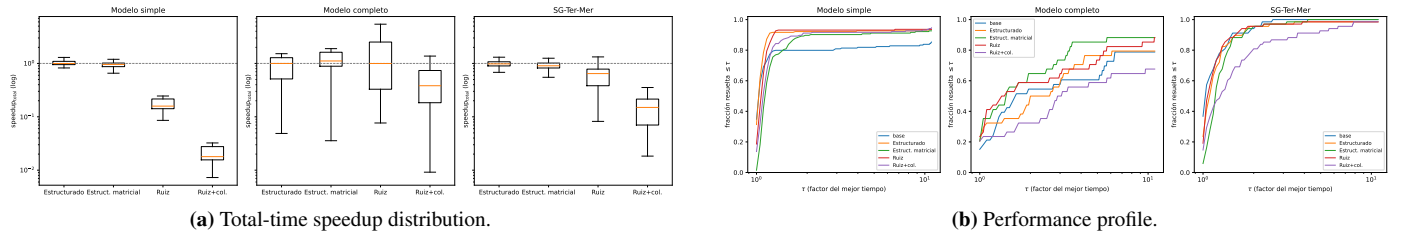


Figure 7: At 1% MIPGap, the CPLEX distributional and performance-profile views break the Gurobi pattern. The plots therefore serve as a solver contrast, not as evidence of universal acceleration.

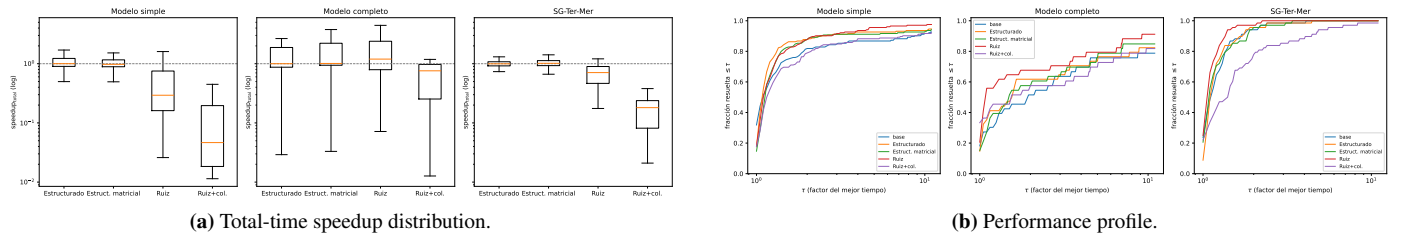


Figure 8: At 0.1% MIPGap, the CPLEX visual evidence remains mixed under the stricter tolerance, supporting a solver-dependent interpretation rather than a robust CPLEX speedup claim.

Conversely, on *none*, scaling provides no mechanistic advantage and can slightly worsen the metric, which is the desired negative control.

The equivalence checks remain clean. Across all 240 synthetic instances, the maximum relative spread of the objective value across variants is 4.3×10^{-4} , below the solver

Table 12: At 0.1% MIPGap, CPLEX remains close to parity for the generator-aware variants, confirming solver dependence.

Class	Variant	n	sgm T	med. sp. ^{det}	PAR10	PI
Simple	Base	204	5.14	1.00	213.6	1.2×10^{-12}
	SA-Aug	204	4.20	1.02	188.4	6.5×10^{-13}
	SA-Mat	204	5.11	1.00	213.5	1.2×10^{-12}
	Ruiz	204	4.13	1.00	192.0	1.0×10^{-15}
	Ruiz+Cols	204	5.28	0.94	546.6	7.7×10^{-13}
Full	Base	33	421.59	1.00	9162.0	2.7×10^{-8}
	SA-Aug	34	422.62	1.07	8930.4	2.7×10^{-8}
	SA-Mat	33	421.02	1.00	9161.6	2.7×10^{-8}
	Ruiz	34	285.10	1.28	5857.3	1.9×10^{-8}
	Ruiz+Cols	33	377.41	0.96	1.1×10^4	5.2×10^{-8}
SG-Ter-Mer	Base	68	2.82	1.00	18.8	1.1×10^{-10}
	SA-Aug	68	2.84	1.01	20.6	1.0×10^{-10}
	SA-Mat	68	2.80	1.00	18.4	1.1×10^{-10}
	Ruiz	68	2.61	1.02	7.5	1.1×10^{-10}
	Ruiz+Cols	68	4.02	0.85	539.5	1.3×10^{-10}

Notes. Bold values mark the best descriptive value within each class and metric. Under CPLEX, these descriptive minima do not imply a robust positive external-scaling effect.

Table 13: Paired CPLEX tests do not support a robust positive runtime effect for the generator-aware variants.

Scenario	Class	Variant	W	p	q_{BH}	Cliff's δ
1% MIPGap	Simple	SA-Aug	9377.0	0.2016	0.3456	-0.058
1% MIPGap	Simple	SA-Mat	7460.0	0.0004	0.0023	+0.125
1% MIPGap	Full	SA-Aug	172.0	0.3357	0.4284	+0.032
1% MIPGap	Full	SA-Mat	131.0	0.1042	0.2083	-0.153
1% MIPGap	SG-Ter-Mer	SA-Aug	1068.0	0.5211	0.5211	-0.003
1% MIPGap	SG-Ter-Mer	SA-Mat	740.0	0.0082	0.0326	+0.055
0.1% MIPGap	Simple	SA-Aug	8765.0	0.0453	0.1213	-0.024
0.1% MIPGap	Simple	SA-Mat	9515.0	0.3173	0.4760	-0.005
0.1% MIPGap	Full	SA-Aug	173.0	0.7140	0.7789	-0.001
0.1% MIPGap	Full	SA-Mat	159.0	0.9368	0.9368	-0.014
0.1% MIPGap	SG-Ter-Mer	SA-Aug	1085.0	0.5908	0.7089	-0.021
0.1% MIPGap	SG-Ter-Mer	SA-Mat	944.0	0.1617	0.3235	-0.012

Notes. Tests are paired against the base formulation on matched instances. Under CPLEX, the effect signs are small, inconsistent, or unfavorable. They do not support a robust positive generator-aware runtime effect.

MIPGap 10^{-3} , and no instance exceeds that threshold. In original scale, the maximum row violation is 1.3×10^{-6} and the maximum integrality violation is 2.0×10^{-6} , with no infeasible run. The synthetic benchmark therefore supports the numerical mechanism-controlled imbalance, conditioning recovery, and

preservation of feasibility and objective values—while making no claim about runtime speedup.

Taken together, the four operational scenarios and the controlled synthetic check support a limited but useful interpretation. Under Gurobi, the structured preprocessing

Table 14: The best descriptive variant depends on the solver pipeline, not only on the external scaling rule.

Scenario	Simple	Full	SG-Ter-Mer	Interpretation
Gurobi, 1%	SA-Mat	SA-Aug	SA-Mat	The two generator-aware rules dominate descriptively; SA-Mat is strongest in Simple and SG-Ter-Mer, while SA-Aug is strongest in Full.
Gurobi, 0.1%	SA-Mat	SA-Aug	SA-Mat	The stricter gap preserves the same pattern and strengthens the evidence that a matrix-only structural rule can be competitive under Gurobi.
CPLEX, 1%	SA-Aug	SA-Mat	Base	Descriptive winners vary by class; paired tests do not support a robust positive generator-aware effect and some signs are unfavorable.
CPLEX, 0.1%	Ruiz	Ruiz	Ruiz	Ruiz is descriptively strong in shifted geometric time, especially in Full, but this is a CPLEX-pipeline-specific pattern rather than evidence for universal Ruiz dominance.

Notes. The table summarizes the lowest shifted geometric solver time within each class. It is descriptive only; the statistical evidence is reported separately in [tables 9](#) and [13](#).

Table 15: Controlled synthetic benchmark: median condition numbers before scaling and after each variant, with worst original-scale violations across the selected cells.

Pattern	S	κ^{pre} (Base)	κ^{post} (SA-Aug)	κ^{post} (SA-Mat)	κ^{post} (Ruiz)	κ^{post} (Ruiz+Cols)	v_{row}	v_{int}
none	0	1.34×10^3	1.09×10^3	2.49×10^3	1.88×10^3	1.88×10^3	4.55×10^{-13}	0
local	3	1.84×10^8	1.07×10^3	4.68×10^3	3.15×10^3	3.15×10^3	2.91×10^{-11}	0
local	6	1.49×10^{14}	1.07×10^3	4.02×10^3	3.67×10^3	3.67×10^3	1.49×10^{-8}	0
coupling	3	6.86×10^5	1.22×10^3	2.49×10^3	2.11×10^3	2.11×10^3	1.75×10^{-10}	0
coupling	6	4.87×10^8	1.22×10^3	2.49×10^3	2.67×10^3	2.67×10^3	1.12×10^{-7}	2.03×10^{-6}
mixed	3	2.06×10^8	1.18×10^3	4.68×10^3	3.24×10^3	3.24×10^3	1.16×10^{-10}	0
mixed	6	1.52×10^{14}	1.22×10^3	4.02×10^3	4.58×10^3	4.58×10^3	1.31×10^{-6}	1.28×10^{-7}

Notes. Values are medians by pattern, severity, and variant, except for the two violation columns, which report worst original-scale violations in the corresponding pattern–severity cell. The grid contains 20 seeds per cell. The synthetic benchmark is a conditioning and equivalence check, not a runtime benchmark.

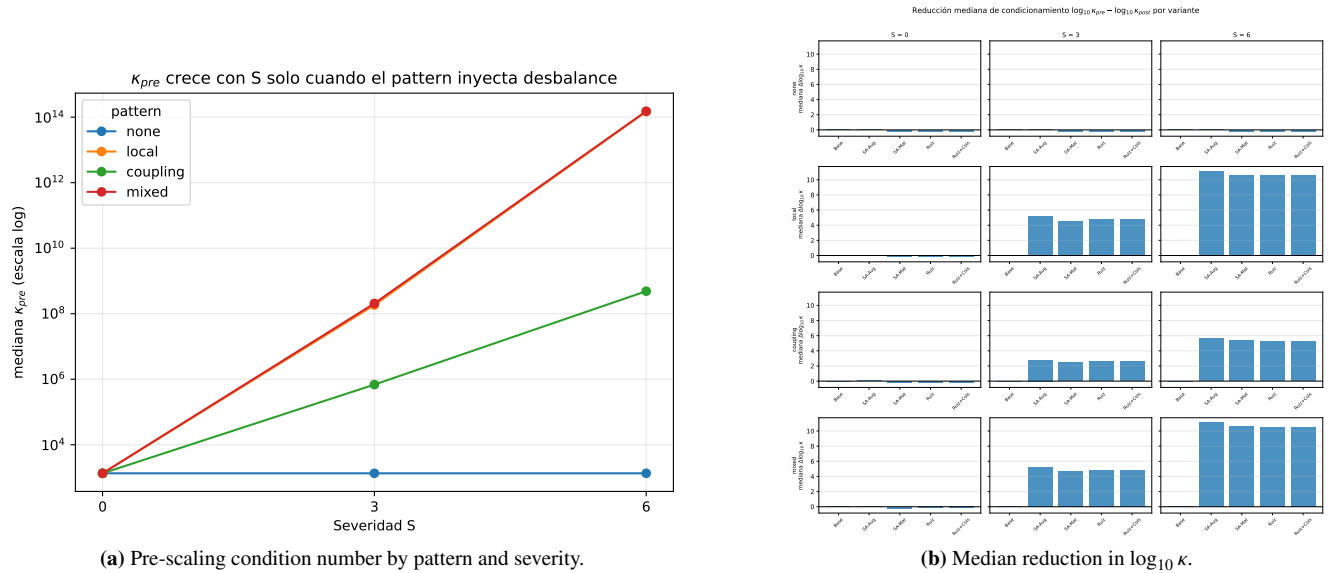


Figure 9: Controlled synthetic mechanism check. Panel (a) verifies that the injected scale imbalance is controlled by the pattern and severity parameters. Panel (b) shows the median conditioning recovery obtained by the scaling variants.

rules improve the numerical representation and often reduce solver time or deterministic effort. Under CPLEX, the same

external transformations do not produce a robust runtime benefit and can be indistinguishable from, or worse than, the base in

Table 16: Claim, evidence, and limitation are distinct parts of the argument.

Claim	Evidence in this study	Limitation
Algebraic equivalence	Positive factors multiply complete constraints; post-solve checks evaluate feasibility, integrality, and objectives in the original scale.	This claim concerns the mathematical MIP, not the path followed by branch-and-bound.
Numerical representation	Generator-aware variants reduce solver-facing conditioning diagnostics in the harder classes.	Conditioning diagnostics describe representation; they do not prove faster MIP search.
Gurobi performance	SA-Aug and SA-Mat improve robust runtime or deterministic-effort metrics in the main Gurobi scenarios.	The strongest variant depends on the instance class, and the claim is conditional on the tested pipeline.
Solver dependence	CPLEX remains near parity or deteriorates in paired runtime evidence, even when some numerical diagnostics improve.	The result is tied to CPLEX 22.1.2.0, default settings, and the ClusterUY environment.
Generalization	Generator-level block information can guide numerical preprocessing before export; the synthetic benchmark injects local and coupling imbalance by construction and confirms the intended conditioning response.	Runtime evidence still comes from one hydrothermal dispatch platform; the synthetic benchmark is a mechanism check, not a performance benchmark.

The table keeps the main characters of the argument visible: the algebraic transformation, the numerical representation, the solver pipeline, and the scope of the evidence.

paired runtime evidence, even when some numerical diagnostics improve. The empirical message is therefore not that one scaling rule always wins. It is that generator-level structure can matter for MIP performance, but its value has to be measured through the full solver–model interaction.

7. Discussion

The experiments support a bounded claim. Generator-aware positive row scaling changes the solver-facing representation in the intended direction. Under Gurobi, that change often reduces robust runtime or deterministic effort. Under CPLEX 22.1.2.0, the same external scaling does not produce a robust runtime gain. The evidence therefore separates three issues that are easy to conflate: algebraic equivalence, numerical representation, and mixed-integer search performance.

Table 16 prevents the main claim from becoming too strong. The method does not promise universal acceleration. It gives a reproducible way to use block and coupling information before export, and it tests whether each solver pipeline benefits from that representation.

A first objection is that commercial solvers already perform scaling internally. This is precisely why internal scaling and presolve are left enabled in the main experiments. The comparison is not external scaling versus a numerically naive solver. It is the realistic question of whether a generator can improve the model representation before the solver’s own pipeline begins. The mixed Gurobi–CPLEX outcome is therefore informative: the same external representation can help one solver pipeline and be absorbed or neutralized by another.

A second objection is that any benefit may come from generic coefficient compression rather than from generator information. The results only partly support that objection. Ruiz-type baselines sometimes compress coefficients and can be descriptively competitive in isolated summaries, but their matched behavior is irregular. The generator-aware rules add a modeling distinction that flat-matrix equilibration does not use: they treat

local rows before coupling rows, and they scale coupling rows relative to the blocks they connect. The evidence therefore points to structure, not only coefficient range, as a source of useful numerical information.

A third objection is that the right-hand side may be unnecessary. The evidence supports this objection more strongly. SA-Aug uses coefficients and right-hand sides and remains strongest in the Full class. Yet SA-Mat recovers much of the benefit using the coefficient matrix alone and is often best in Simple and SG-Ter-Mer. In this testbed, the local–coupling distinction appears more important than including the right-hand side in every scale proxy. Other generated models may behave differently, but the present evidence does not need the right-hand side to explain the main Gurobi gains.

A fourth objection comes from CPLEX. Several transformations improve static or solver-level numerical diagnostics, yet paired runtime evidence remains neutral, mixed, or unfavorable. This contrast does not weaken the numerical claim; it limits the performance claim. A commercial MIP run is not a linear algebra experiment. Presolve, internal scaling, relaxation bases, cuts, incumbents, branching, heuristics, tolerances, and deterministic scheduling all stand between the exported matrix and the final runtime. External scaling may improve one part of that pipeline while the binding bottleneck lies elsewhere.

The main methodological lesson is therefore about evidence. Numerical preprocessing for MIPs should not be evaluated with one coefficient-range statistic or one runtime table. Algebraic equivalence shows that the transformed model is legitimate. Numerical diagnostics show that the representation changed in the intended direction. Matched runtime evidence shows whether the solver converted that representation into speed. Solver contrasts show whether the effect survives a different internal pipeline. The study contributes by reporting all four layers and by keeping their claims separate.

The controlled synthetic benchmark closes one part of this gap. It shows that when the source of scale imbalance is known by construction, the scaling variants respond in the intended

direction and preserve original-scale feasibility and objective values. It does not close the performance gap: the synthetic instances are small and solve in milliseconds, so runtime claims must still come from operational instances. A further operational ablation would be useful but is not needed for the present scoped claim. The real-data comparison separates right-hand-side information and generic equilibration, but it does not run local-only, coupling-only, and local-plus-coupling versions on the hydrothermal instance classes. Such an ablation would refine attribution by identifying how much of the Gurobi effect comes from local row normalization and how much comes from the coupling-row treatment. Additional operational domains, such as logistics, production planning, and network design, would test whether the same mechanism persists when the generator structure itself changes. Until then, the supported conclusion is scoped: generator-aware positive row scaling is a numerical preprocessing layer for generated block-structured engineering MIPs, not a solver-independent acceleration theorem.

8. Strengths, limitations, and reproducibility

The study is strongest where the evidence is matched. Runtime comparisons use the same instance, solver, and tolerance for the base and scaled variants. Numerical diagnostics are reported separately from speed. Feasibility and integrality are checked on the original scale. These choices make the argument auditable: a reader can see which evidence supports algebraic equivalence, which evidence supports numerical improvement, and which evidence supports solver-dependent acceleration.

The main limitation is domain coverage. The method applies to generated modular MIPs with identifiable local blocks and coupling rows, but the evidence comes from one hydrothermal dispatch platform. That platform supplies a realistic and numerically heterogeneous testbed. It does not prove that the same behavior will appear in other generated optimization domains.

A second limitation is component-level attribution within the operational testbed. The synthetic benchmark isolates local and coupling imbalance by construction, but the hydrothermal experiments compare the full generator-aware architecture against right-hand-side and generic-scaling alternatives. They do not include operational local-only, coupling-only, and local-plus-coupling variants on the real hydrothermal classes. This omission does not affect algebraic equivalence, and it does not invalidate the observed solver-dependent performance pattern. It does limit causal attribution: the current evidence identifies the value of the complete local–block–coupling architecture, not the separate marginal contribution of each stage.

Solver dependence is another threat. The experiments leave internal presolve and scaling enabled, so the measured effect is the effect of an external preprocessing layer inside realistic commercial solver pipelines. That design makes the experiments relevant to practice, but it also makes the results conditional on solver version, settings, and environment. The negative CPLEX evidence should therefore be read as a finding about the tested pipeline, not as a universal statement about CPLEX.

The numerical diagnostics have limited reach. Exported coefficient ranges are static matrix summaries. Solver-level conditioning diagnostics depend on the state exposed by the solver and on auxiliary fixed-integer LP computations. Neither quantity measures full MIP difficulty. For that reason, the paper reports original-scale feasibility, integrality after descaling, objective consistency checks, performance profiles, deterministic effort, paired tests, and primal–dual progress as complementary evidence.

Finally, the experiments use operational data. Some raw inputs may not be redistributable. The reproducibility package should therefore include public code, aggregate results, paired-instance identifiers, processed logs, and scripts that rebuild the tables and figures. The controlled synthetic benchmark in [sections 5.2 and 6.5](#) provides an auditable complement when raw operational data cannot be released: it can reproduce the numerical mechanism without claiming operational speedup.

9. Conclusion

Generated engineering MIPs contain useful structure before export. The generator knows the local rows, the component blocks, and the coupling rows. A solver usually receives only the flat matrix. This paper uses that pre-export information to choose positive row scalings that preserve the optimization problem but change its numerical representation.

The answer to the research question is qualified but affirmative. Generator-aware positive row scaling improves solver-facing numerical diagnostics for the tested block-structured hydrothermal MIPs. Under Gurobi, that improvement often coincides with lower robust runtime or deterministic effort. Under CPLEX 22.1.2.0, the same external transformations do not produce a robust runtime gain. The method improves representation; acceleration remains solver-dependent.

The main contribution is the local–block–coupling architecture. The method scales local rows first, summarizes local blocks second, and scales coupling rows last with the scales of the blocks they connect. This order uses information that the model generator knows but a generic flat-matrix routine may not. Because each factor multiplies a complete constraint by a strictly positive scalar, feasible points, objective values, and integer restrictions remain unchanged.

The broader lesson is evidential. A claim about numerical preprocessing for generated MIPs needs more than coefficient ranges. It needs algebraic equivalence, original-scale feasibility checks, solver-facing numerical diagnostics, matched runtime comparisons, and solver-contrast evidence. On that chain of evidence, the study supports a scoped claim: generator-aware positive row scaling is useful for the tested class of generated engineering MIPs, but it is not a universal acceleration mechanism.

Future work should test the mechanism beyond hydrothermal dispatch and should add an operational ablation of the architecture itself. The controlled synthetic benchmark introduced here already isolates local and coupling imbalance in a reproducible setting; larger synthetic suites and local-only, coupling-only, and

local-plus-coupling runs on real generated models would clarify which stage of the architecture drives the observed solver effects. Additional generated MIP applications in logistics, production planning, and network design can test whether the same mechanism persists when the engineering structure changes. The reproducibility package should separate implementation-specific effects, large external artifacts, and the numerical mechanism studied here.

Author Contributions

Mathias Rodríguez Castro: Conceptualization, Methodology, Software, Investigation, Formal analysis, Data curation, Visualization, Writing – original draft, Writing – review and editing.

Funding

This research received no specific grant from any funding agency in the public, commercial, or not-for-profit sectors.

Conflicts of Interest

The author declares no conflicts of interest.

Ethics Statement

This study consists of computational experiments and does not involve human participants, animal subjects, patient data, or clinical-trial data.

Data Availability Statement

The reproducibility package for this study is publicly available at <https://github.com/MathiasRodriguezCastro/generator-aware-row-scaling>. The repository contains the cleaned experimental source code, preprocessing scripts, the redistributable dispatch input instances used in the paper, the synthetic benchmark instances, aggregate result tables, paired-instance identifiers, and the scripts used to rebuild the reported tables and figures. Large per-instance solver logs and bulky dispatch CSVs are not kept in the Git tree only to keep the repository lightweight; they are distributed as external archives linked from the repository's `artifacts/README.md`. The repository also documents the solver versions, scenario definitions, command-line options, time limits, MIPGap settings, preprocessing flags, and log-parsing rules used to construct the matched comparisons. The aggregate results were produced by the original ClusterUY runs, while the released C++ code is a cleaned reproducibility version containing the monolithic MIP path and preprocessing methods used in the paper.

Supporting Information

Additional processed solver logs, extended tables, diagnostic plots, and scripts for rebuilding the numerical evidence are available in the reproducibility repository or as external artifacts linked from it.

Acknowledgments

The author acknowledges Universidad de la República and the computational resources used for the experimental evaluation. The experiments presented in this paper were carried out using ClusterUY (<https://cluster.uy>) [11]. The author thanks Claudio Risso and Ignacio Ramírez for supervision, methodological discussions, and feedback during the development of this work. The MIP formulation used in this study belongs to a broader line of work on hydroelectric and hydrothermal dispatch models developed by Claudio Risso and collaborators.

References

- [1] C. Risso, S. Nesmachnow, R. Porteiro, J. M. Vignolo, E. Sierra, M. Ibarburu, A mixed combinatorial optimization model for the río negro hydroelectric complex, in: Proceedings of the VII Ibero-American Congress of Smart Cities, ICSC-Cities 2024, Springer Proceedings in Energy, Springer, Singapore, 2025, pp. 403–420, presented at ICSC-Cities 2024, San Carlos, Costa Rica.
- [2] C. Risso, S. Nesmachnow, R. Porteiro, J. M. Vignolo, E. Sierra, M. Ibarburu, A mixed combinatorial optimization model to manage the hydrothermal dispatch for the uruguay river hydroelectric plant, in: ICSC-CITIES 2025 – VIII Congreso Ibero-Americano de Ciudades Inteligentes, Puebla, México, 2025, pp. 1–15.
- [3] M. Rodríguez Castro, M. E. Montañó Cobas, E. Zampetti Garin, Implementación de modelo de despacho eléctrico para el complejo hidroeléctrico del río negro, Tesis de grado, Universidad de la República, Facultad de Ingeniería, Instituto de Computación, Montevideo, Uruguay (2025).
- [4] A. J. Wood, B. F. Wollenberg, G. B. Sheblé, Power Generation, Operation, and Control, 3rd Edition, Wiley, 2013.
- [5] A. J. Conejo, M. Carrión, J. M. Morales, Decision Making Under Uncertainty in Electricity Markets, Springer, 2010. doi:10.1007/978-1-4419-7421-1.
- [6] J. P. Vielma, Mixed integer linear programming formulation techniques, SIAM Review 57 (1) (2015) 3–57. doi:10.1137/130915303.
- [7] E. Klotz, A. M. Newman, Practical guidelines for solving difficult linear programs, Surveys in Operations Research and Management Science 18 (1–2) (2013) 1–17. doi:10.1016/j.sorms.2012.11.001.
- [8] E. Klotz, A. M. Newman, Practical guidelines for solving difficult mixed integer linear programs, Surveys in Operations Research and Management Science 18 (1–2) (2013) 18–32. doi:10.1016/j.sorms.2012.11.002.
- [9] D. Ruiz, A scaling algorithm to equilibrate both rows and columns norms in matrices, Tech. Rep. RAL-TR-2001-034, Rutherford Appleton Laboratory (2001).
- [10] E. D. Dolan, J. J. Moré, Benchmarking optimization software with performance profiles, Mathematical Programming 91 (2) (2002) 201–213. doi:10.1007/s101070100263.
- [11] S. Nesmachnow, S. Iturriaga, Cluster-uy: Collaborative scientific high performance computing in uruguay, in: M. Torres, J. Klapp (Eds.), Supercomputing, Vol. 1151 of Communications in Computer and Information Science, Springer, Cham, 2019.